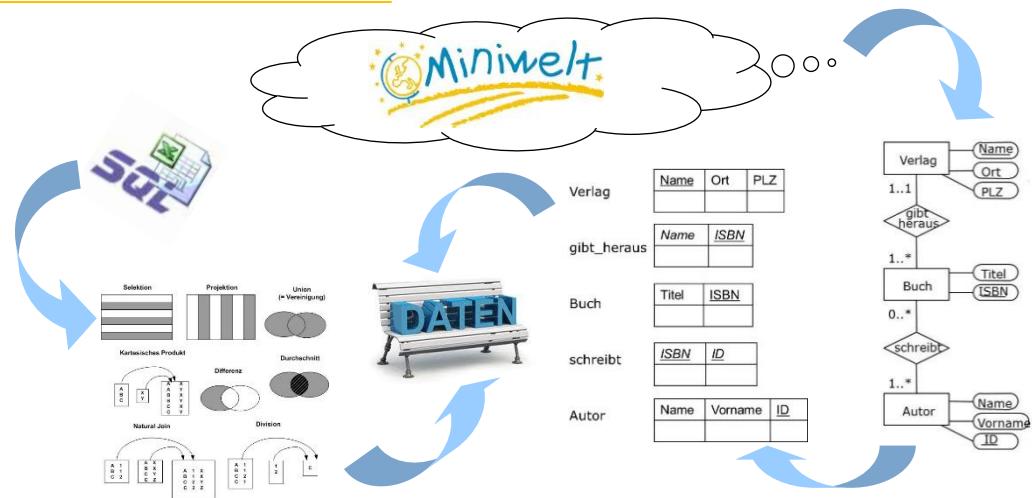


Modul 1

Grundlagen Datenmodellierung und Datenbanksysteme

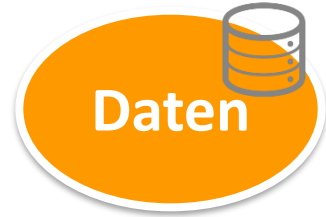
Josef Altmann



Überblick

- Einführung und Motivation
- Aufgaben von Datenbanksystemen
- Historische Entwicklung
- Entwicklungstrends
- Anhang I - IV: Definitionen

Einführung



- "Ein **Datenbanksystem** (DBS) ist ein **System** zur **elektronischen Datenverwaltung**." [Wikipedia]

- D.h., **elektronische Daten** sollen u.a.

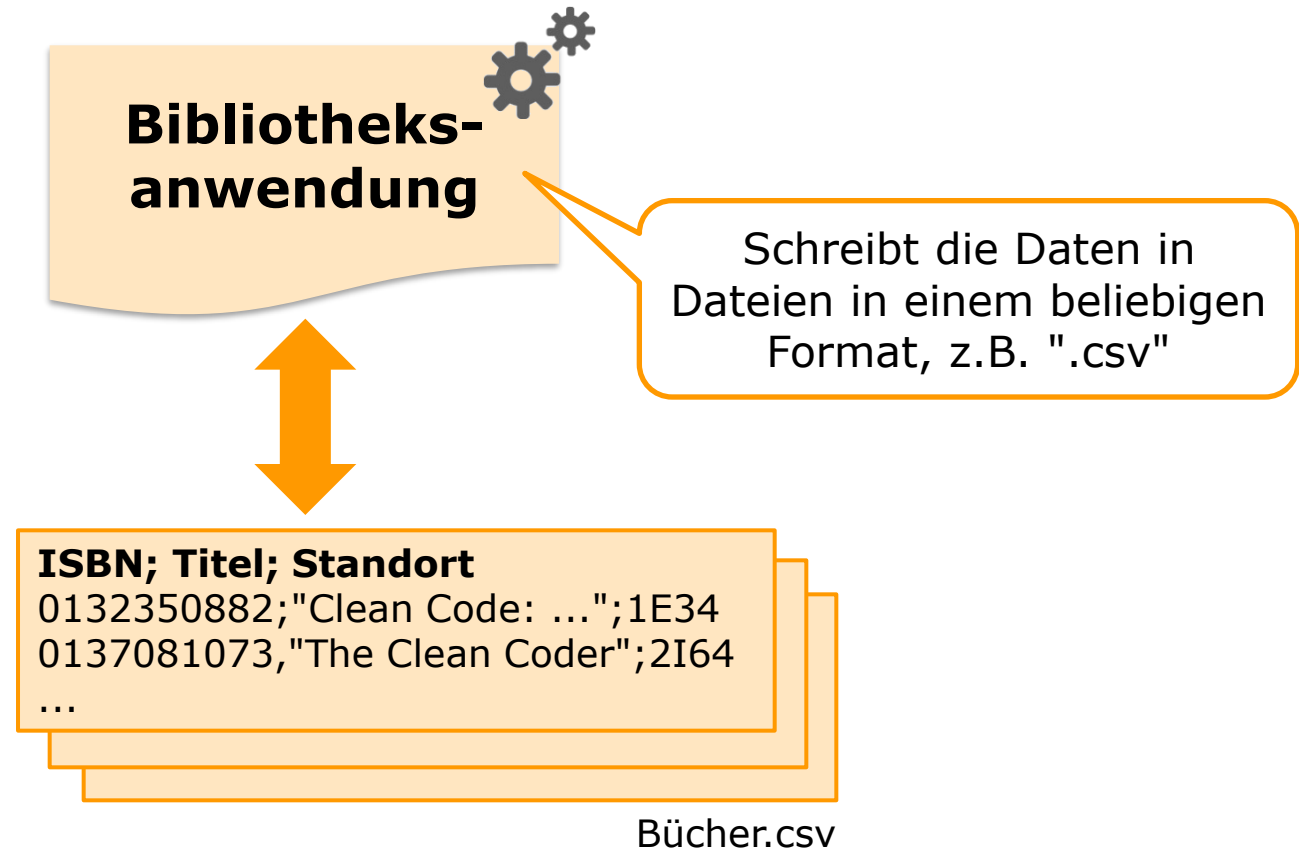
- **erfasst, verändert, gelöscht**
- wieder **erlangt** (analysiert) und
- dauerhaft **gespeichert** (persistiert) werden können

ABER: das könnte man doch auch mit **Dateien** realisieren, oder?!?

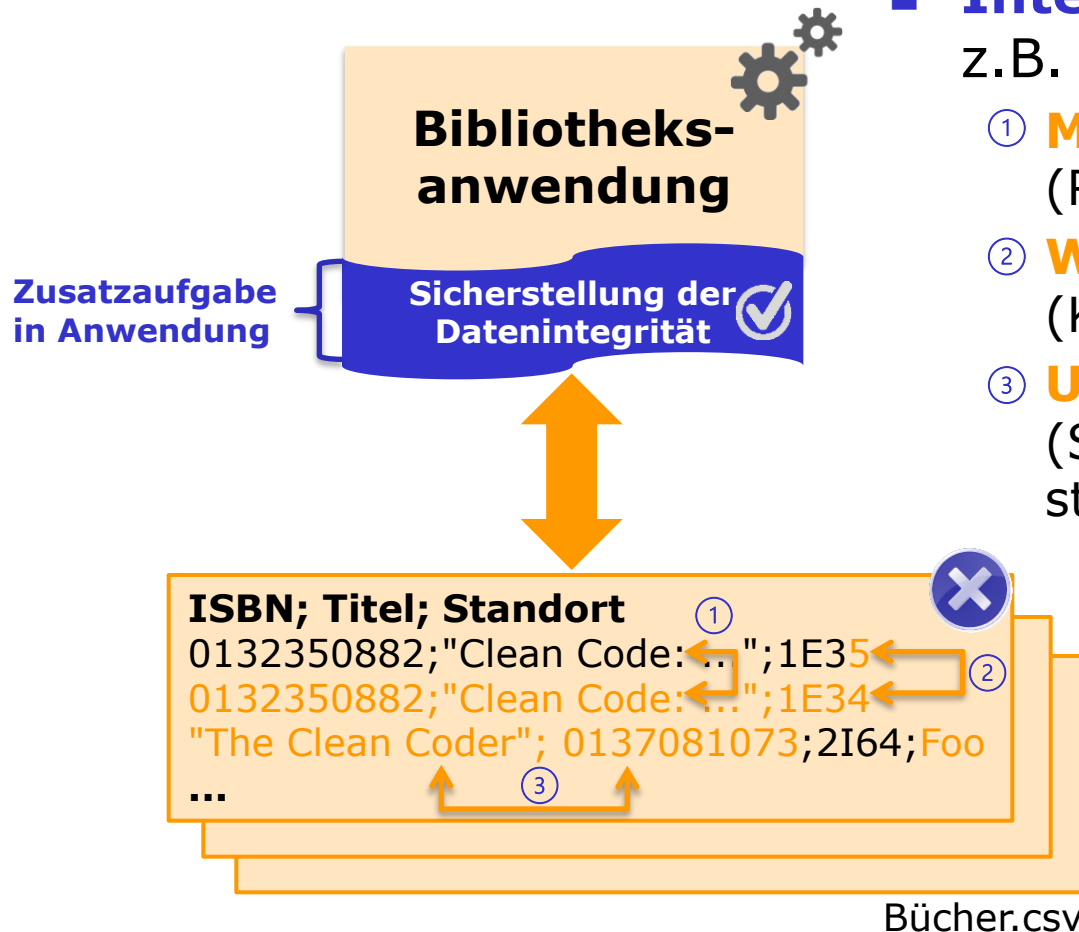
- **Beispiel Bibliotheksanwendung**

- Bücher sollen **erfasst** werden können; der Standort von Büchern kann sich **ändern**; falls Bücher ausscheiden, sollen sie **gelöscht** werden
- Man möchte **abfragen** können, wo sich ein bestimmtes Buch befindet bzw. ob es aktuell verfügbar ist
- Die Buchdaten sollen **langfristig** zur Verfügung stehen

Datenhaltung mit Dateien



Probleme bei Datenhaltung mit Dateien

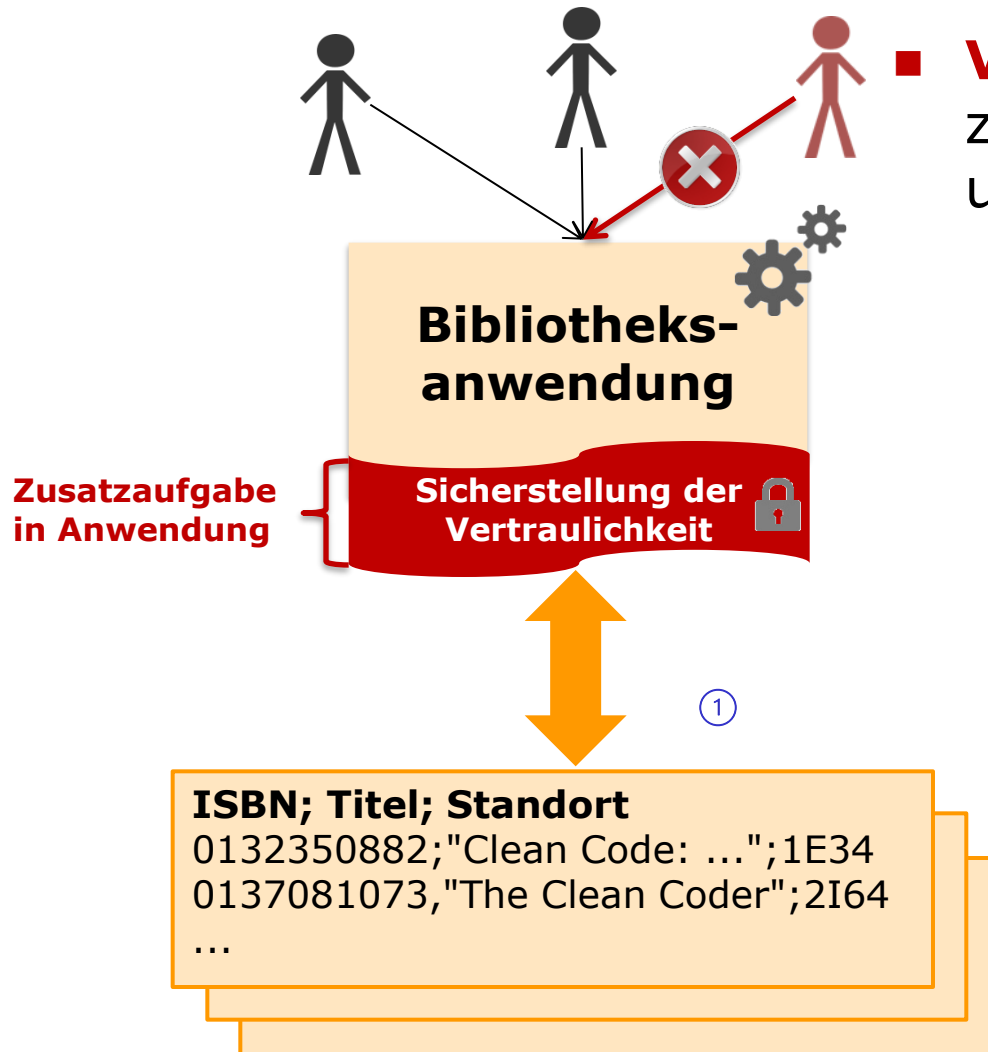


■ Integrität der Daten?

z.B. Vermeidung von

- ① **Mehrfachspeicherung** (Redundanz)
- ② **Widersprüchlichen Daten** (Konsistenz)
- ③ **Unterschiedlichem Aufbau** (Struktur), z.B. in **Anzahl** (4 statt 3) bzw. **Art** (z.B. Datentyp)

Probleme bei Datenhaltung mit Dateien



■ Vertraulichkeit der Daten?

z.B. Schutz der Daten vor unberechtigtem Zugriff, d.h.,

- **unberechtigte Personen** sollen nicht zugreifen dürfen bzw.
- bestimmte Personen dürfen u.U. nur einen **Ausschnitt** der **Daten** sehen

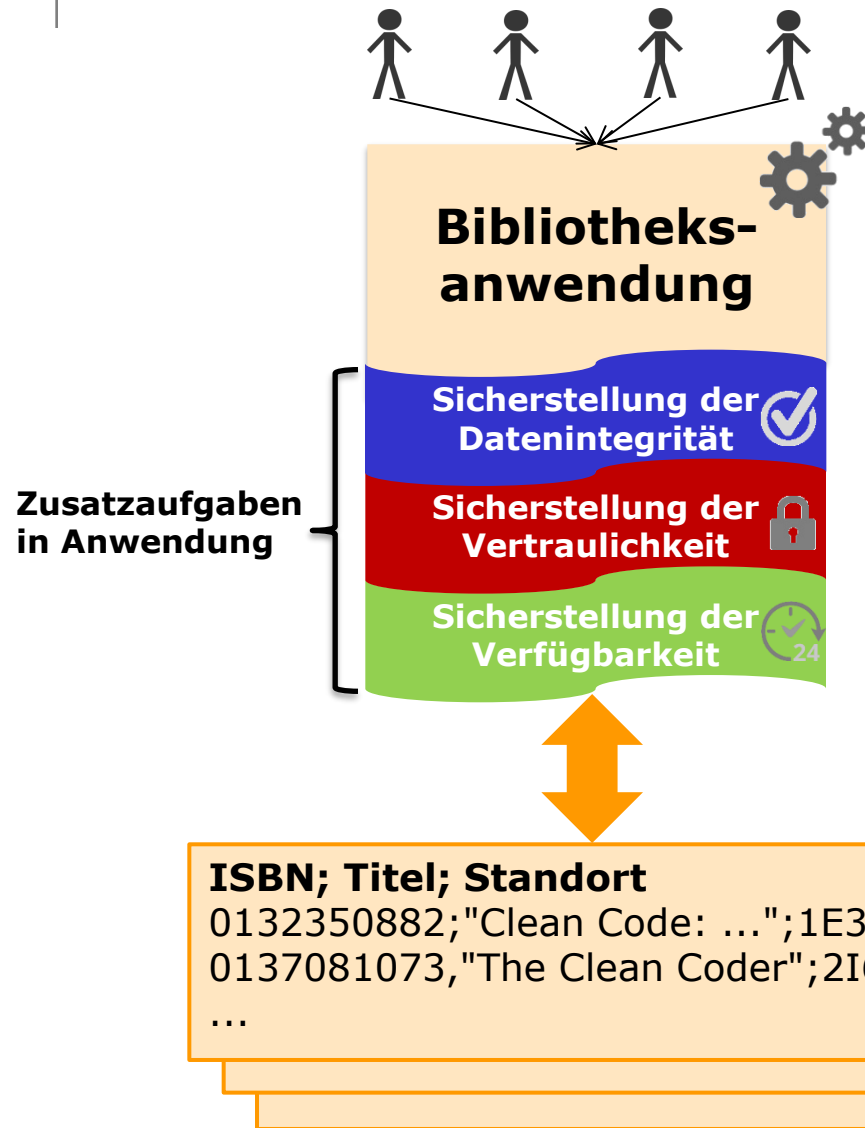
Bücher.csv

Probleme bei Datenhaltung mit Dateien



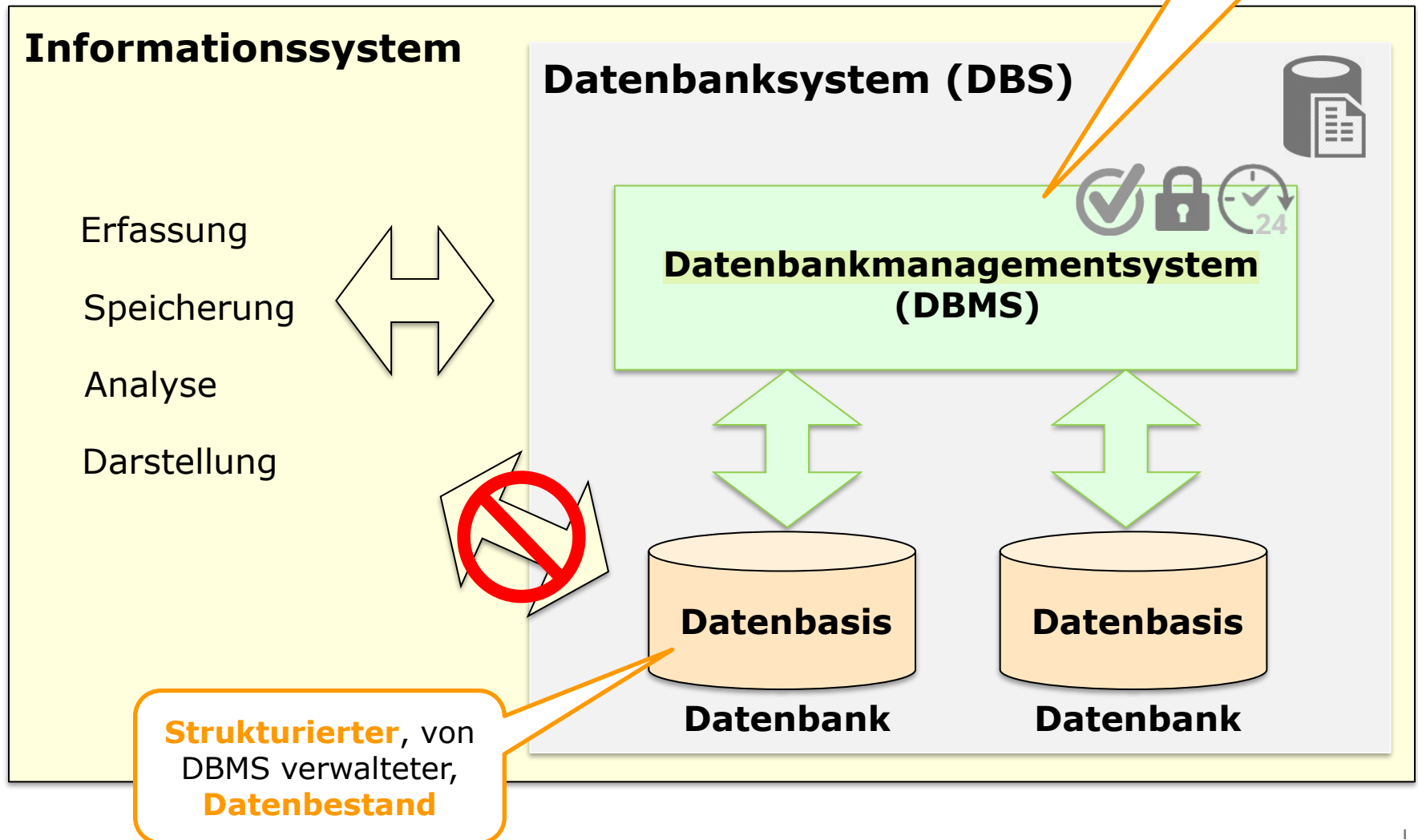
- **Verfügbarkeit der Daten?**
z.B. auch im **Fehlerfall**, d.h.,
bei einem Stromausfall oder
Systemcrash

Probleme bei Datenhaltung mit Dateien



- D.h., jede Anwendung, die Daten verwaltet, müsste zusätzlich **Datenmanagementaufgaben** berücksichtigen → **erheblicher Implementierungsaufwand**
- Aufgaben **wiederholen** sich von Anwendung zu Anwendung; d.h., sind **anwendungsunabhängig**

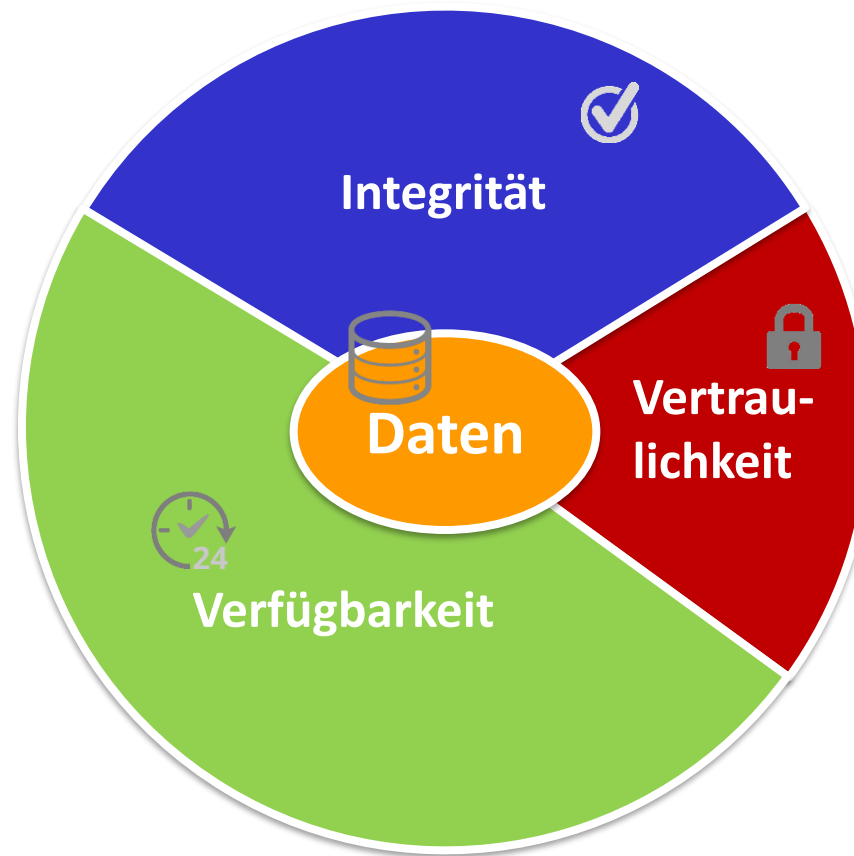
Begriffsabgrenzungen

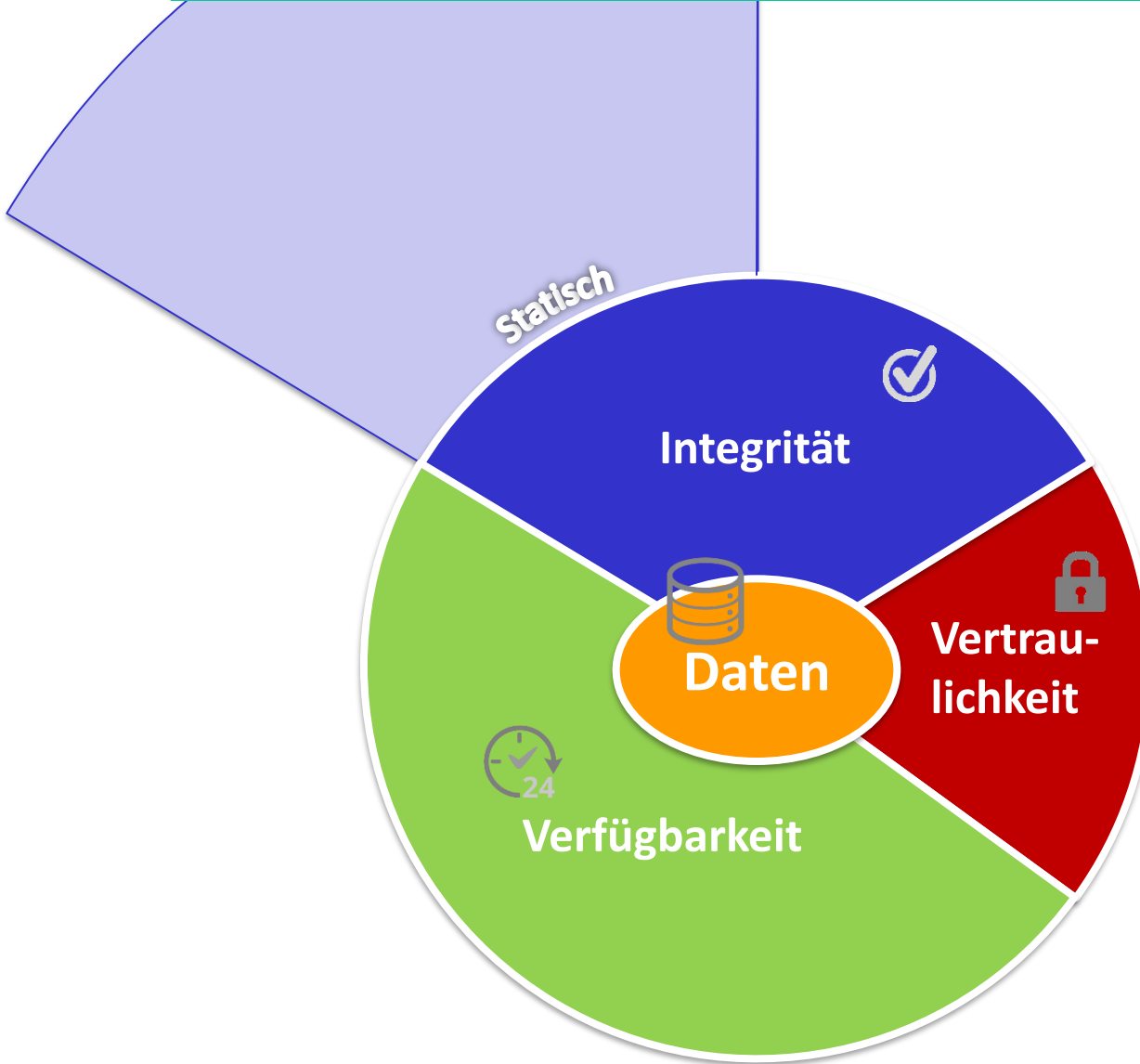


Überblick

- Einführung und Motivation
- Aufgaben von Datenbanksystemen
- Historische Entwicklung
- Entwicklungstrends
- Anhang I - IV: Definitionen

Aufgaben von Datenbanksystemen im Detail





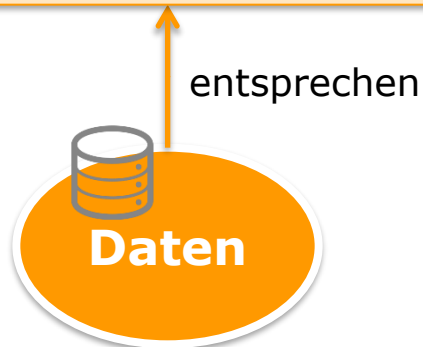
Aufgaben von Datenbanksystemen im Detail

Statische Integrität

■ Integritätsbedingungen

DB-Schema

- = **Integritätsbedingungen** (Regeln), die sicherstellen, dass die Daten in einem **gültigen (konsistenten) Zustand** sind
- Daten, die diesen **Regeln widersprechen**, können **nicht eingefügt** werden

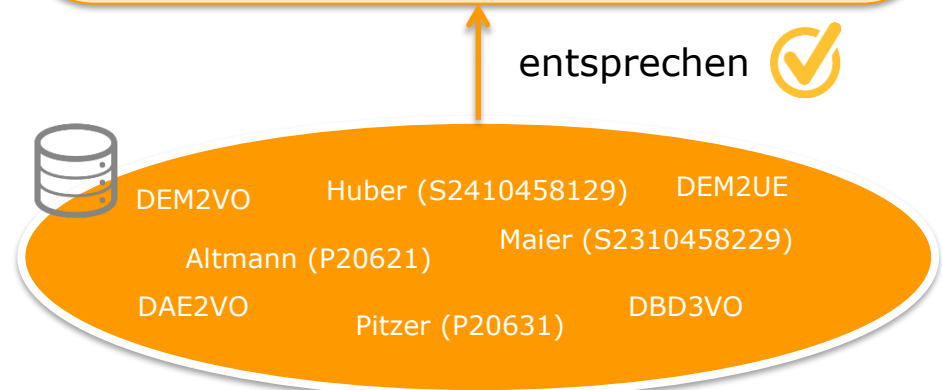


Beispiel

DB-Schema (in Prosa)

- Es gibt **Studenten**, **Professoren** und **Lehrveranstaltungen**
- Studenten** sind durch eine **Matrikelnummer** und einen **Namen** gekennzeichnet ...
- Professoren** sind durch eine **Personalnummer** und einen **Namen** gekennzeichnet ...
- Lehrveranstaltungen** sind durch eine **Lehrveranstaltungsnummer**, **Bezeichnung** und **Lehrveranstaltungstyp** gekennzeichnet ...

Datenbanksysteme bieten **formale Sprache**, um DB-Schemata zu definieren



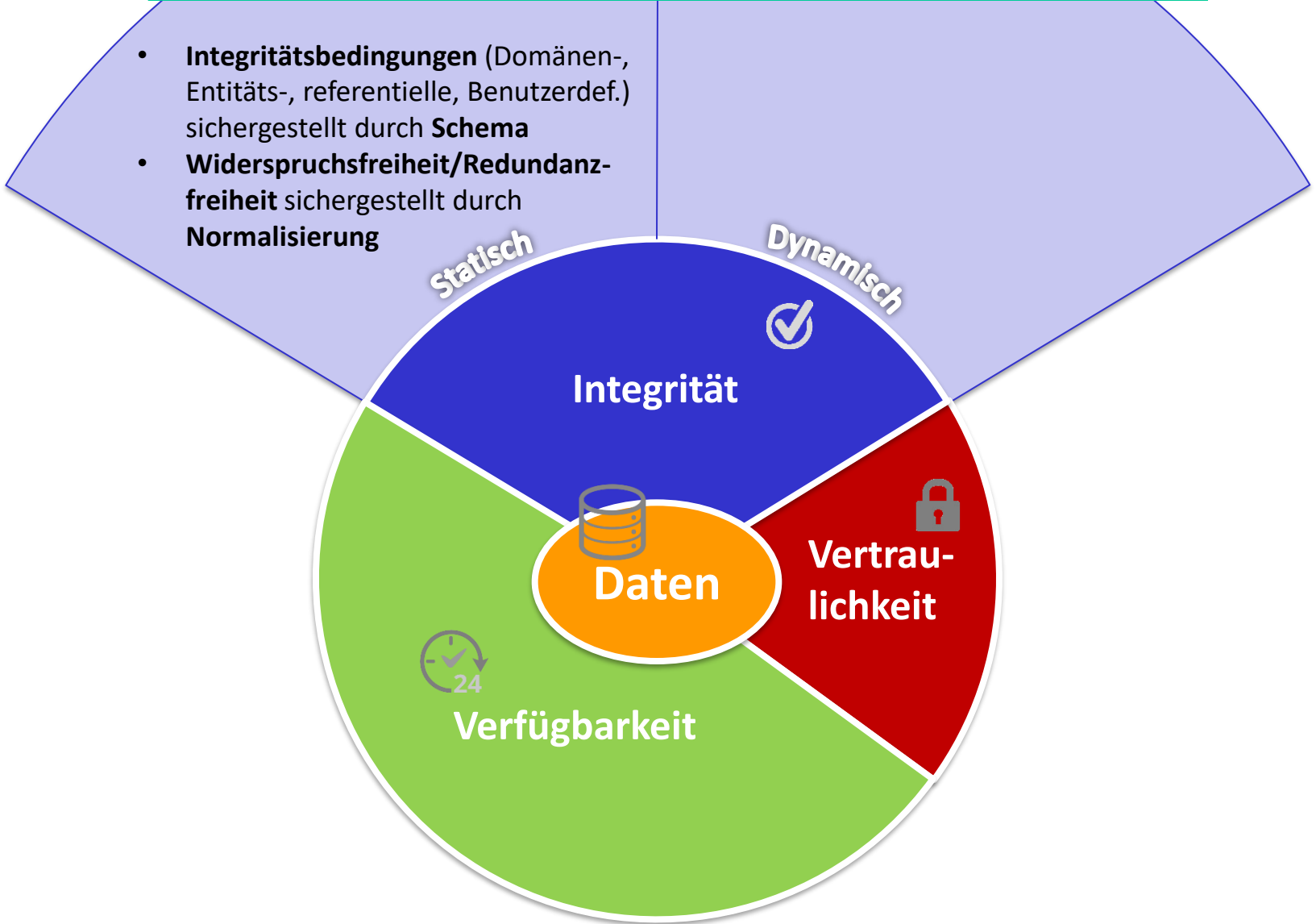
Datenausprägung einer Hochschule

Aufgaben von Datenbanksystemen im Detail

Statische Integrität

■ Widerspruchsfreiheit/Redundanzfreiheit

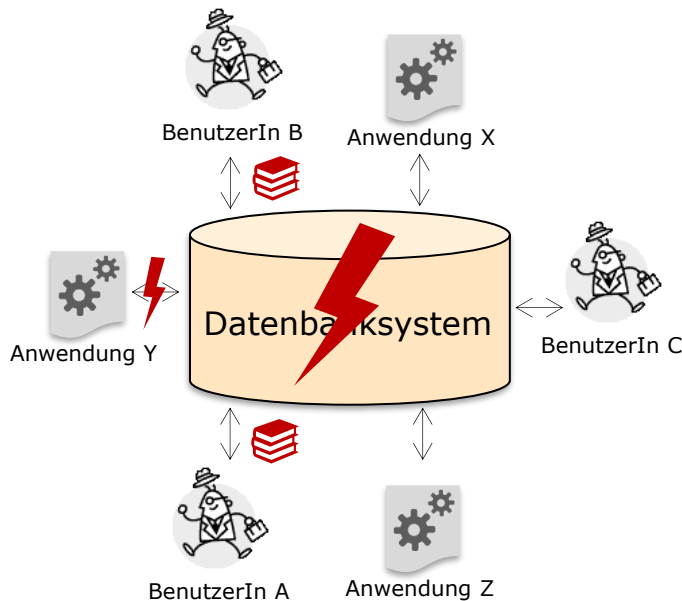
- DB-Schema soll so entworfen sein, dass **Redundanzfreiheit** gewährleistet ist, d.h., **kein Datum doppelt gespeichert** wird
 - Spart **Speicherplatz**
 - Schützt vor **Änderungsanomalien** (gibt nur einen Speicherort, der geändert werden muss → Widersprüche können nicht entstehen)
 - Erlaubt **performantes Einfügen**; Lesen dafür u.U. nicht so performant!
- Um dies zu gewährleisten, gibt es einen wohldefinierten **Prozess** beim **Entwurf** eines DB-Schemas (**Normalisierung**)
- Darüber hinaus bietet das DB-Schema einen gewissen Schutz durch bestimmte **Integritätsbedingungen** (z.B. Entitätsintegrität – keine Entität darf zwei Mal existieren)



Aufgaben von Datenbanksystemen im Detail

Dynamische Integrität

■ In der Realität von Datenbanksystemen kann vieles schiefgehen ...



- Datenbanksoftware oder -hardware kann jederzeit ausfallen (auch mitten in einer Schreiboperation).
- Anwendung kann jederzeit abstürzen (auch mitten in einer Folge von Operationen).
- Störungen im Netzwerk können unerwartet die Anwendung von der Datenbank bzw. vom Datenbankknoten abschneiden.
- Mehrere Clients können gleichzeitig in die Datenbank schreiben und dabei ihre Änderungen gegenseitig überschreiben.
- Ein Client kann Daten lesen, die keinen Sinn ergeben, weil sie nur teilweise aktualisiert wurden
- ...

Aufgaben von Datenbanksystemen im Detail

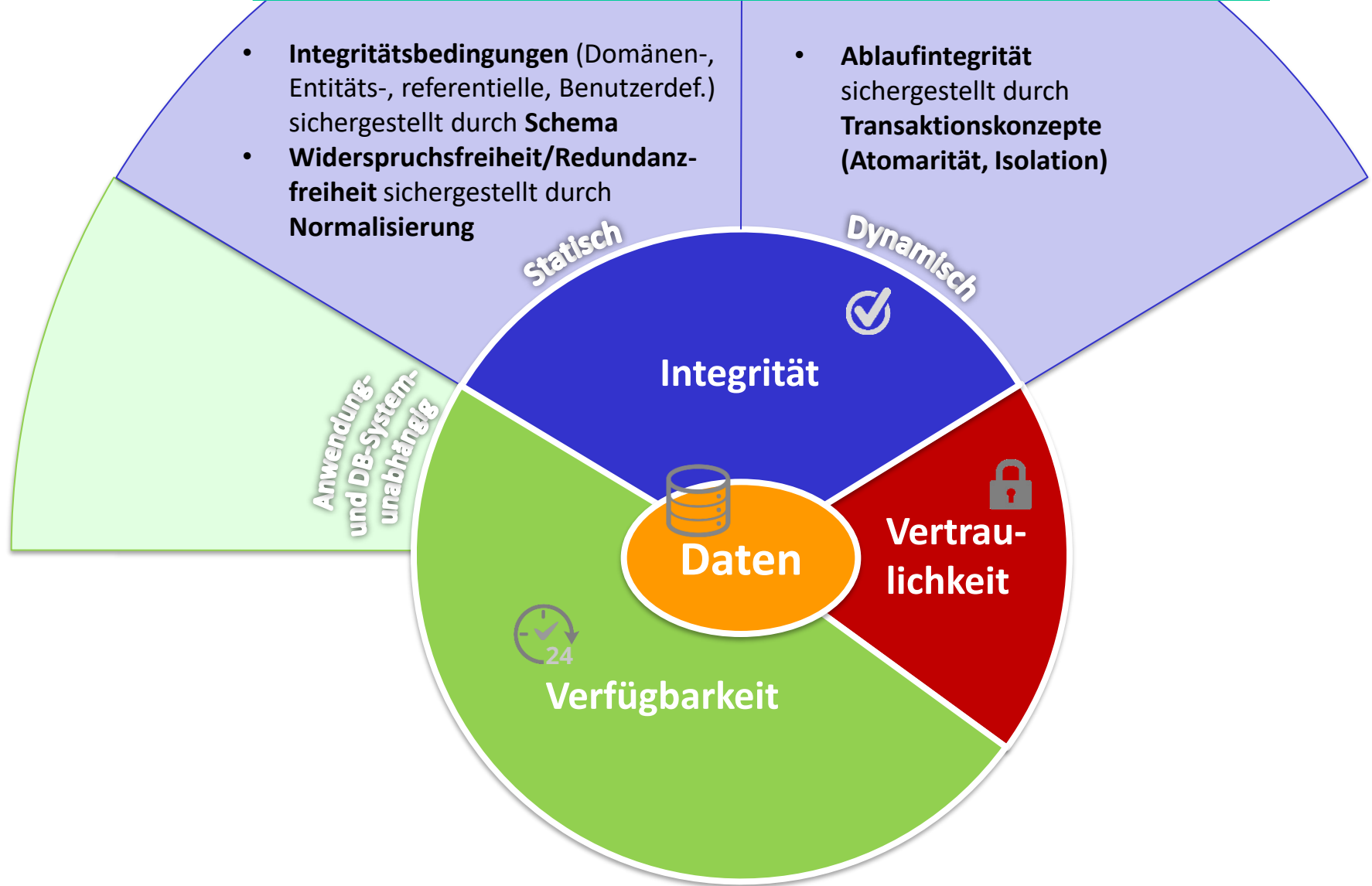
Dynamische Integrität

■ Ablaufintegrität

- Datenbanksystem stellt sicher, dass sich die Daten selbst dann in einem konsistenten Zustand befinden,
 - falls im **zeitlichen Verlauf mehrere Benutzer gleichzeitig** auf die Daten **zugreifen** bzw.
 - falls **mehrere Operationen** logisch zusammenhängen

■ Hierfür **bietet** ein **Datenbanksystem**

- **Isolation** – d.h., eine Datenbank erscheint für einen Benutzer immer so, als ob er der einzige Benutzer wäre; dies wird erreicht, indem verhindert wird, dass sich nebenläufig in Ausführung befindliche Operationen gegenseitig beeinflussen
- **Atomarität** – zusammenhängende Operationen werden immer alle gemeinsam oder gar nicht ausgeführt (Alles-oder-nichts-Eigenschaft)



Aufgaben von Datenbanksystemen im Detail

Anwendungs- und DB-System-unabhängige Verfügbarkeit

■ Relationenmodell

- Klasse der relationalen Datenbanksysteme hält die Daten in einem **standardisierten Format**

■ SQL (Structured Query Language)

- Definition von relationalen DB-Schemata sowie der Zugriff auf die darin gespeicherten Daten erfolgt mittels **standardisierter Sprache**

- Durch diese **standardisierte Schnittstelle** bieten relationale Datenbanksysteme eine **Anwendungs- und DB-System-unabhängige Verfügbarkeit** von Daten

Aufgaben von Datenbanksystemen im Detail

Anwendungs- und DB-System-unabhängige Verfügbarkeit

Beispiel Relationenmodell

Student		hoeren		Vorlesung	
MatrNr	Name	MatrNr	VorlNr	VorlNr	Titel
26120	Fichte	25403	5022	5001	Grundzüge
25403	Jonas	26120	5001	5022	Glaube und Wissen
...	...	...	...	...	...

Beispiel SQL

```

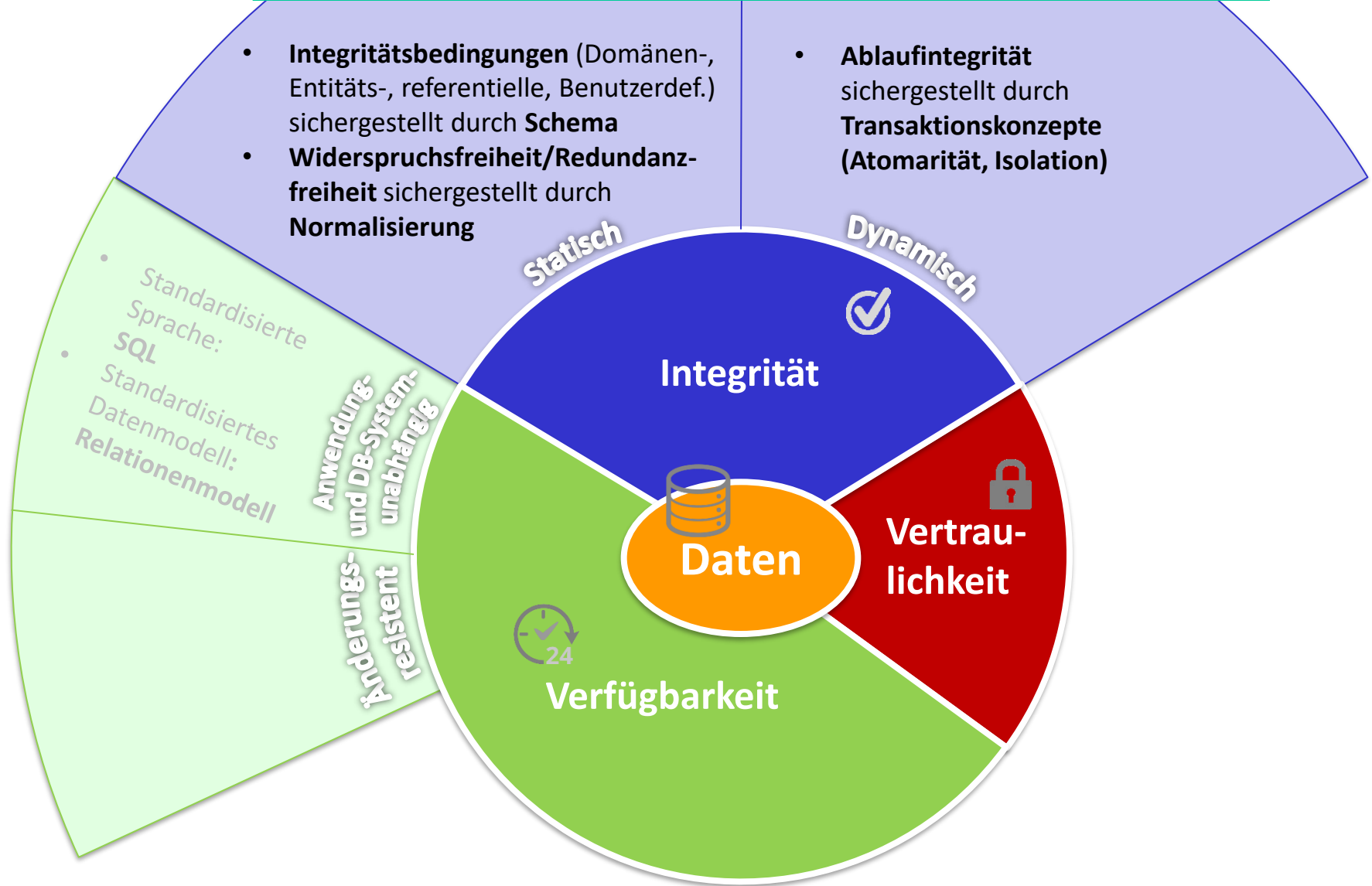
SELECT Name
FROM Student, hoeren, Vorlesung
WHERE Student.MatrNr = hoeren.MatrNr AND
        hoeren.VorlNr = Vorlesung.VorlNr AND
        Vorlesung.Titel = 'Grundzüge';

```

```

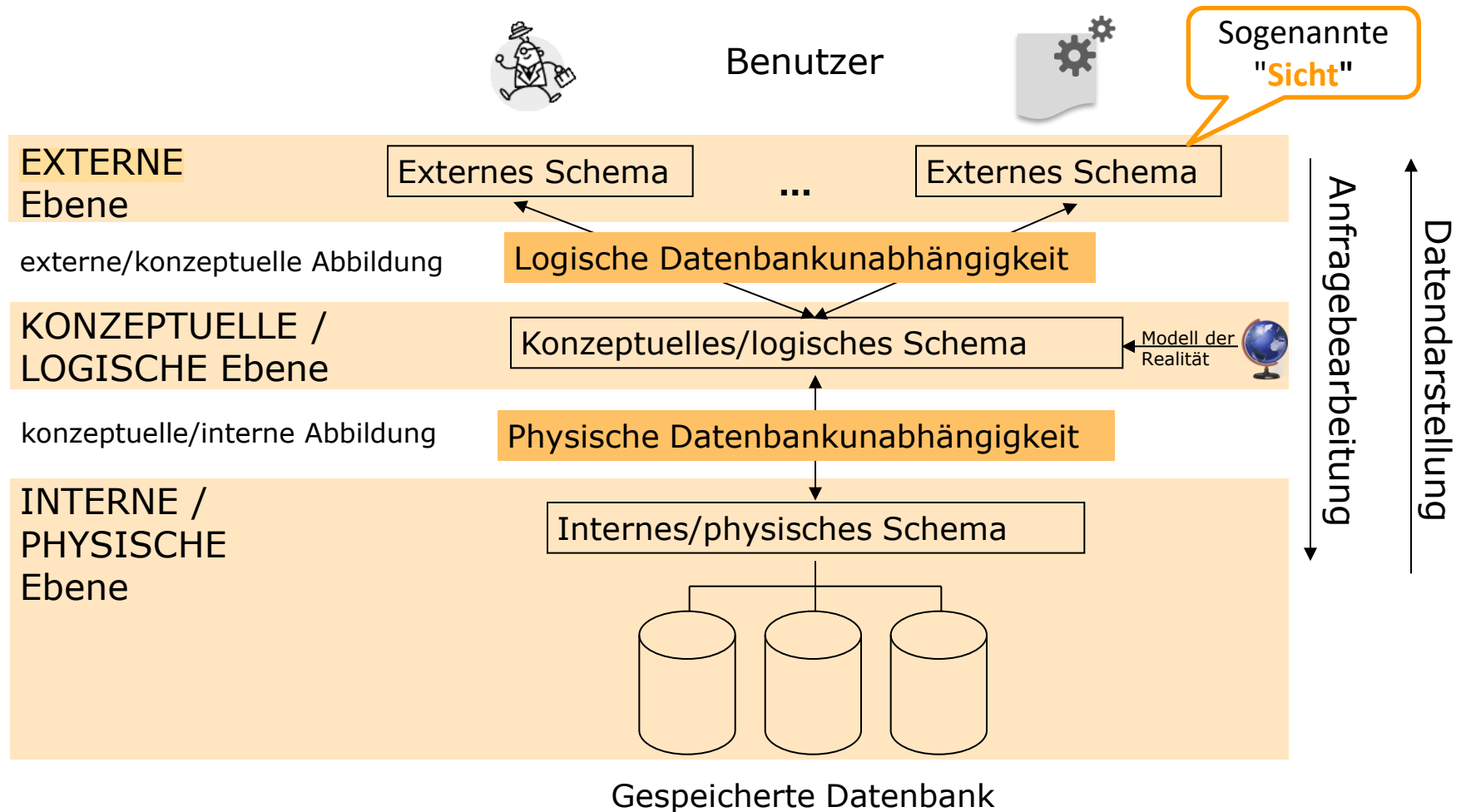
UPDATE Vorlesung
SET Titel = 'Grundzüge der Logik'
WHERE VorlNr = 5001;

```



Aufgaben von Datenbanksystemen im Detail

Änderungsresistente Verfügbarkeit

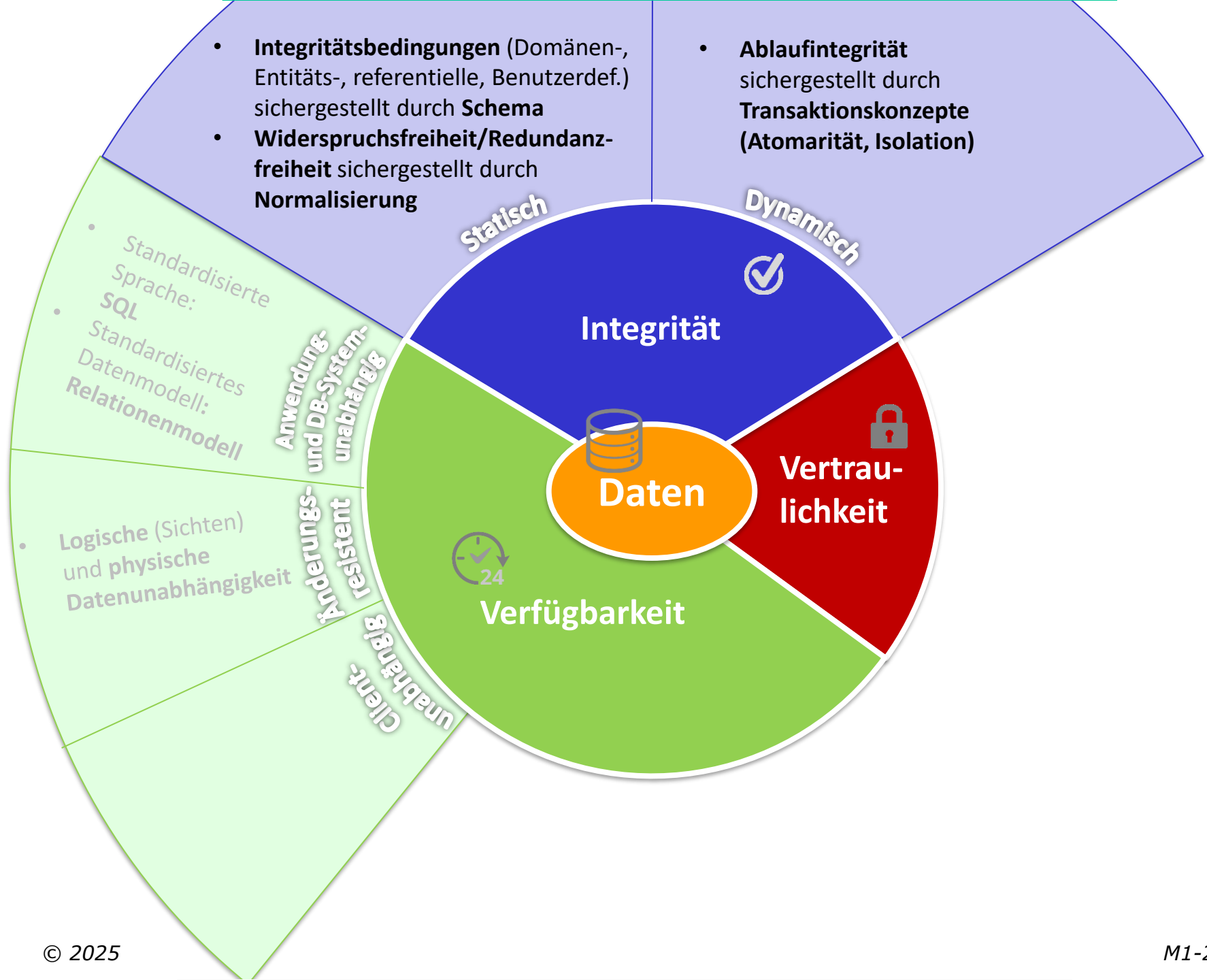


ANSI/X3/SPARC: Normungsvorschlag
für Systemarchitektur von DBMS

Aufgaben von Datenbanksystemen im Detail

Änderungsresistente Verfügbarkeit

- **Physische Datenunabhängigkeit**
 - Abgrenzung der internen Ebene zu den anderen Ebenen
 - Änderungen im physischen Medium haben keine Auswirkungen auf Benutzer
- **Logische Datenunabhängigkeit**
 - Abgrenzung der konzeptionellen Ebene zur externen Ebene
 - Änderungen in der DB-Struktur haben keine Auswirkungen auf Benutzer
- Durch diese "Indirektionsstufen" wird eine soweit als mögliche **änderungsresistente Verfügbarkeit** der Daten erreicht



Aufgaben von Datenbanksystemen im Detail

Client-unabhängige Verfügbarkeit

- Über **unterschiedliche Clients** kann man auf Daten zugreifen bzw. diese manipulieren, z.B.:
 - über dedizierte **Benutzerschnittstellen** und/oder **Entwicklungsumgebungen**, z.B. **Microsoft Excel**, **JetBrains DataGrip**, **VS Code**

The screenshot illustrates two different client environments used to interact with a database. On the left, Oracle SQL Developer shows a query in the Query Builder: `SELECT PersNr, Name, Raum FROM Professor WHERE Raum = 226;`. The result set is displayed below the query, showing one row with values 2125, Sokr..., and 226. On the right, Microsoft Excel is shown with the 'Daten' (Data) ribbon active. The 'Neue Abfrage' (New Query) button is highlighted with a pink arrow. A tooltip for this button is visible, stating: 'Daten aus mehreren Quellen einfach untersuchen, verbinden und kombinieren und dann modellieren und einschränken, sodass sie Ihren Anforderungen entsprechen.' Below the tooltip is a link for 'Weitere Infos' (More Info).

Aufgaben von Datenbanksystemen im Detail

Client-unabhängige Verfügbarkeit

```
package at.fhoee;

import java.sql.*;
import java.util.Properties;

public class Program {
    public static final String URL = "jdbc:postgresql://localhost:5432/postgres";
    public static final String USER = "postgres";
    public static final String PASS = "dae5";
    public static final String SCHEMA = "uni";
    public static final String LOGLEVEL = "OFF"; // OFF, DEBUG, TRACE

    public static void main(String[] args) {
        try {
            // retrieve the PostgreSQL database driver
            Class.forName("org.postgresql.Driver");

            // create properties for the database connection
            Properties connectionProperties = new Properties();
            connectionProperties.setProperty("user", USER);
            connectionProperties.setProperty("password", PASS);
            connectionProperties.setProperty("currentSchema", SCHEMA);
            connectionProperties.setProperty("loggerLevel", LOGLEVEL);

            // get the connection to the database
            Connection conn = DriverManager.getConnection(URL, connectionProperties);

            // create a new statement
            Statement stmt = conn.createStatement();
            String query = "SELECT * from Studenten";

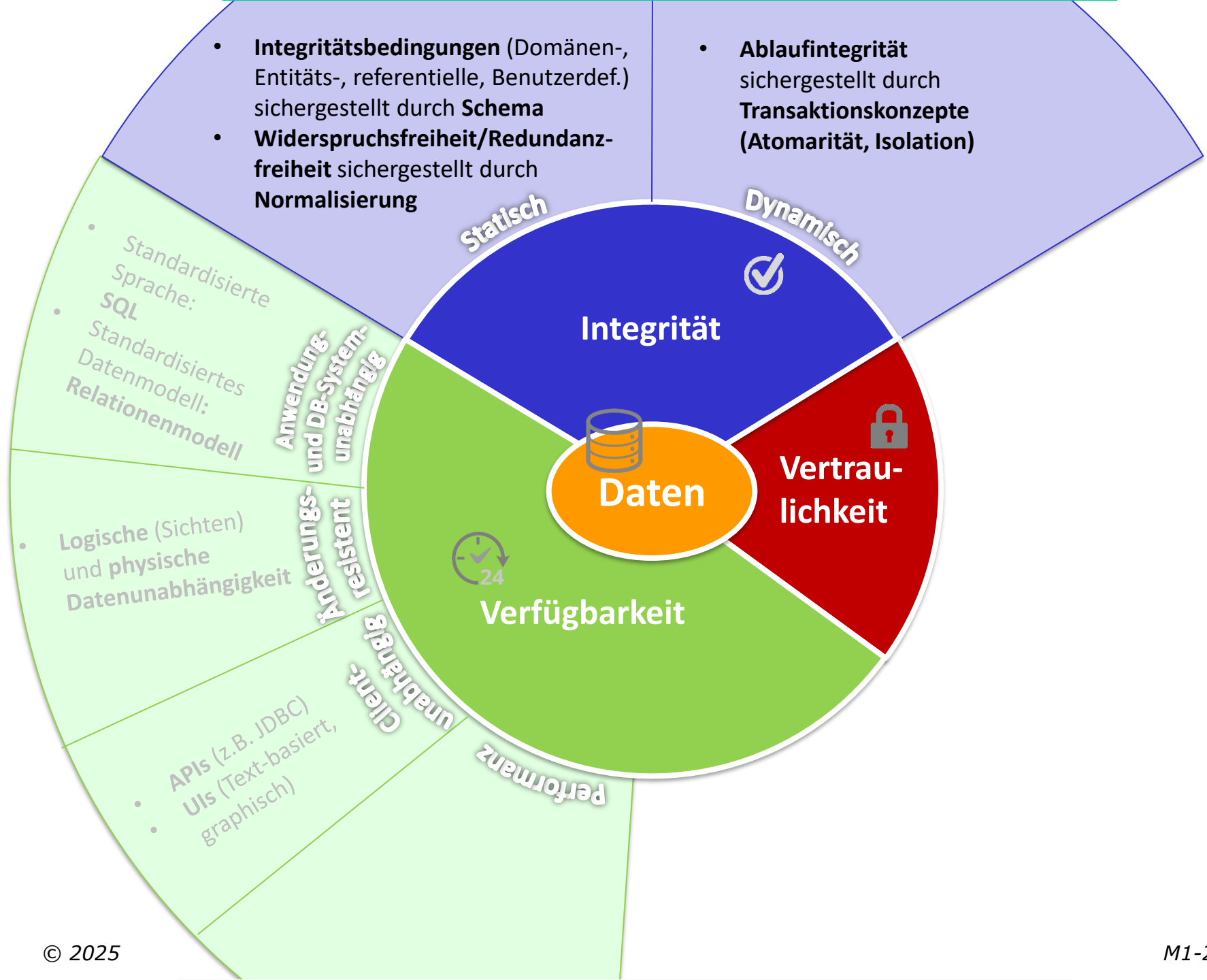
            // execute the statement - return a result set
            ResultSet result = stmt.executeQuery(query);

            // print results
            while (result.next()) {
                String name = result.getString("name");
                String matrNr = result.getString("matrnr");
                System.out.println "[" + matrNr + " ] " + name;
            }

            // close the connection
            conn.close();
        } catch (SQLException sqlException) {
            sqlException.printStackTrace();
        } catch (ClassNotFoundException classNotFoundException) {
            classNotFoundException.printStackTrace();
        }
    }
}
```

- Programmatisch (API), z.B. in Java (JDBC), PHP

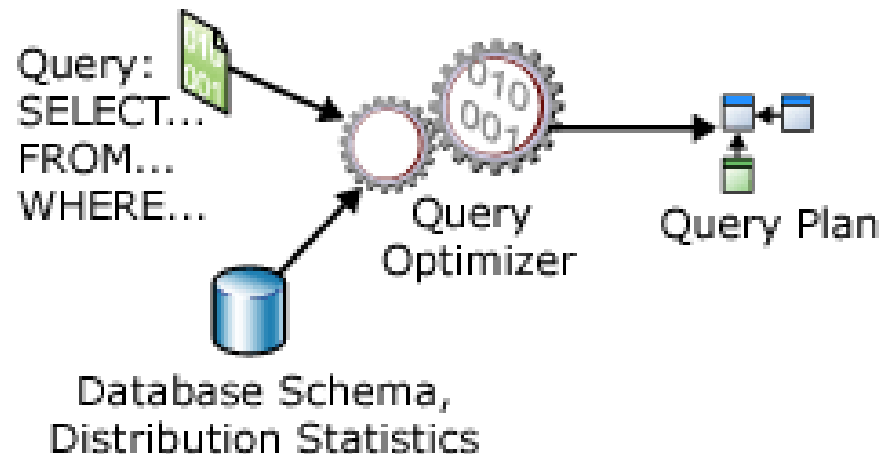
```
Run: Program x
/Library/Java/JavaVirtualMachines/jdk1.8.0_152.jdk/Contents/Home/bin/java ...
[24002] Xenokrates
[25403] Jonas
[26120] Fichte
[26830] Aristoxenos
[27550] Schopenhauer
[28106] Carnap
[29120] Theophrastos
[29555] Feuerbach
Process finished with exit code 0
```



Aufgaben von Datenbanksystemen im Detail

Performante Verfügbarkeit

- Datenbanksysteme bieten diverse Mechanismen, um Daten **performant zugreifbar** zu machen
 - Effiziente Anfrageabarbeitung durch **automatische Anfrageoptimierung**



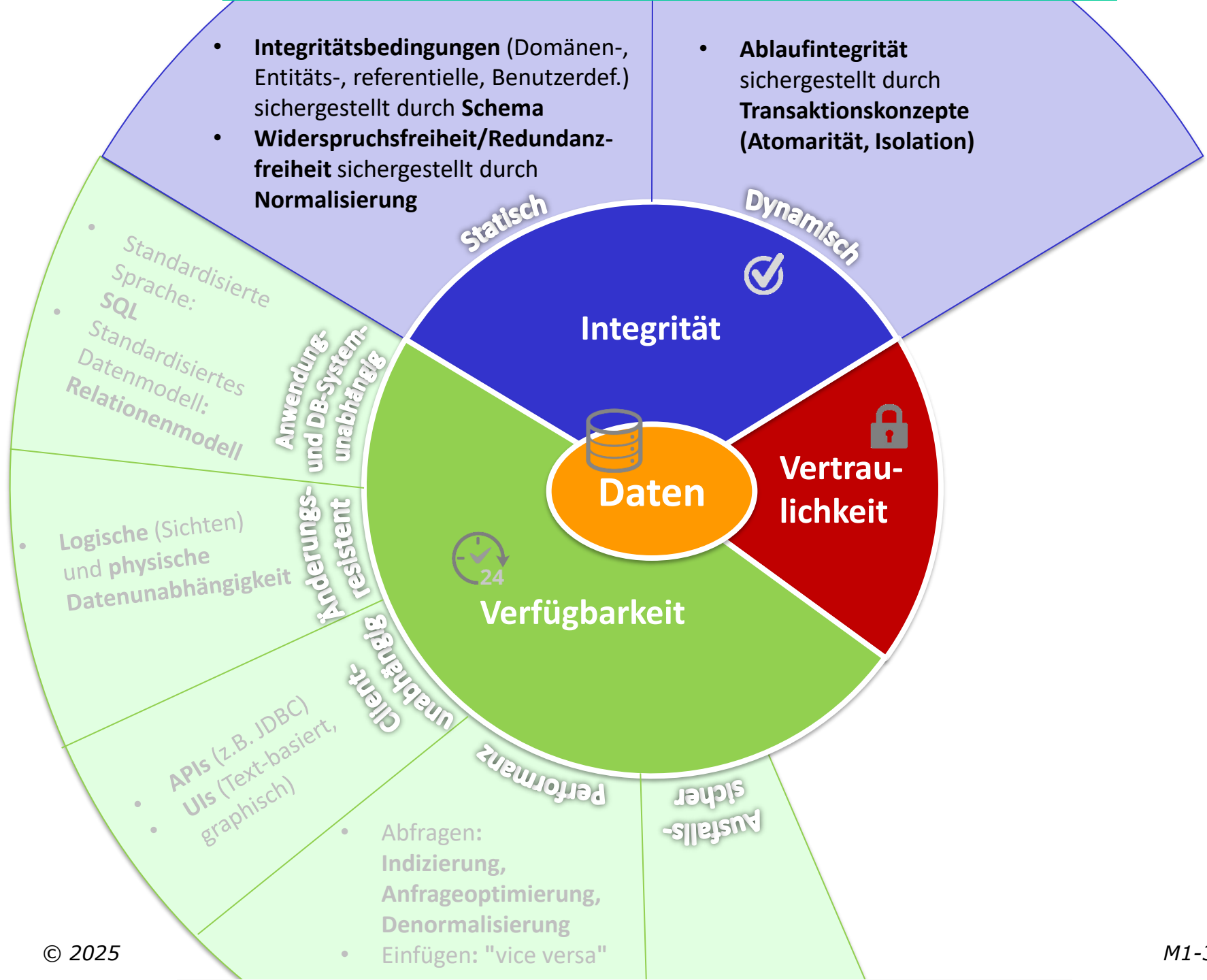
Aufgaben von Datenbanksystemen im Detail

Performante Verfügbarkeit

- Effizienter Datenzugriff durch **Hilfsstrukturen – Indizes** (redundante, gespeicherte sortierte Zugriffspfade)

KNr	Name	...	Ort
4711	Altmann	...	Linz
5588	Schulz	...	Wien
6834	Mayr	...	Wien
...	...	...	...

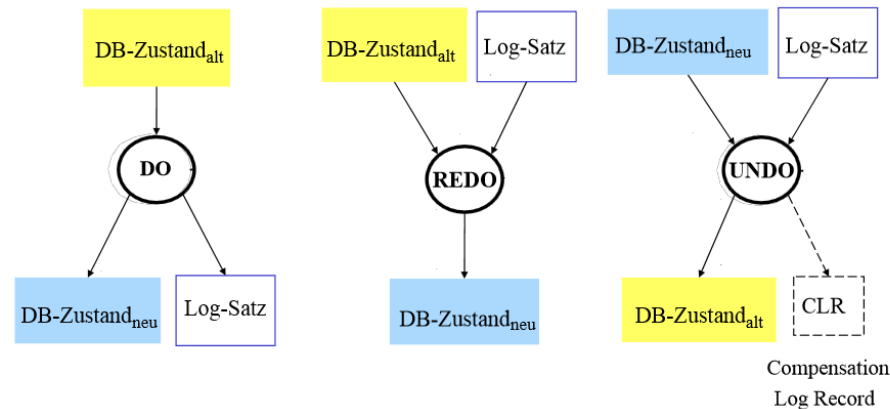


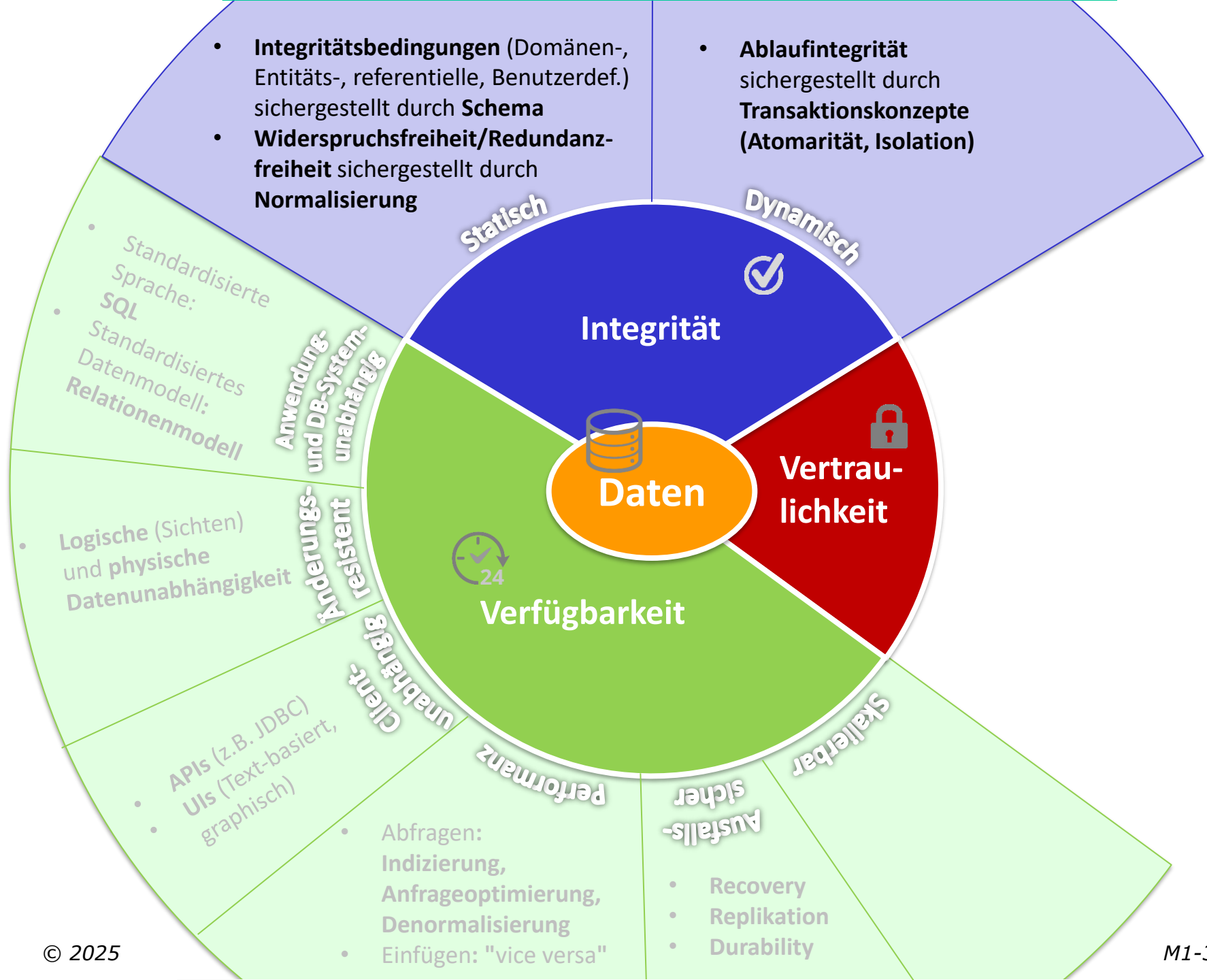


Aufgaben von Datenbanksystemen im Detail

Ausfallssichere Verfügbarkeit

- Datenbanksysteme bieten Mechanismen, um Daten sogar bei **Fehlern** bzw. Ausfällen nach wie vor zur Verfügung zu haben (Recovery)
- **Grundidee**: Verwendung von **Logs**, um wieder einen **konsistenten Zustand** herstellen zu können, z.B. um
 - einen **früheren Zustand** der Daten **wiederherstellen** zu können bzw.
 - einen **späteren Zustand** der Daten **wiederherstellen** zu können

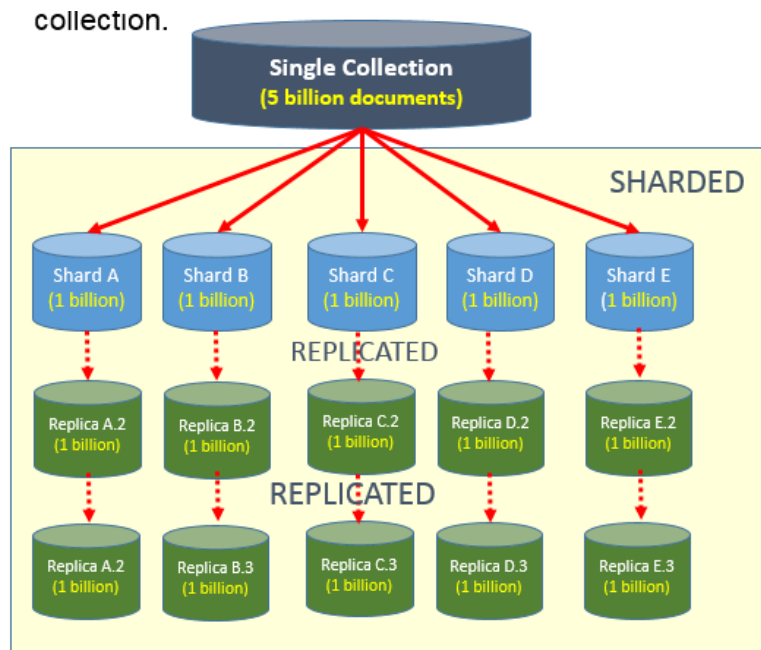


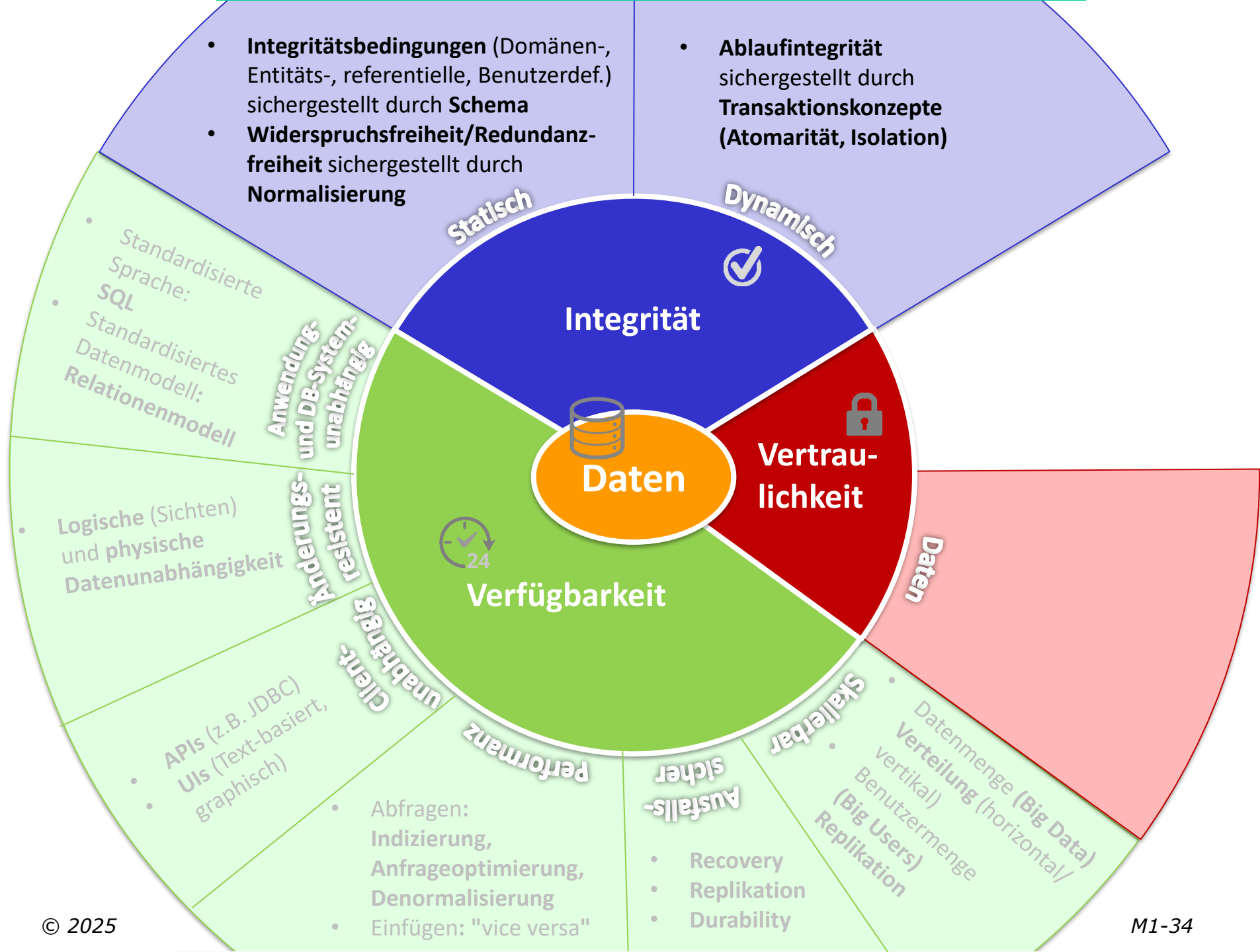


Aufgaben von Datenbanksystemen im Detail

Skalierbare Verfügbarkeit

- Datenbanksysteme bieten Mechanismen, um mit **großen Datenmengen** bzw. großen **Benutzerzahlen** umgehen zu können
- Grundidee: **Verteilung** (Aufteilung) bzw. **Replikation** (Wiederholung) der Daten



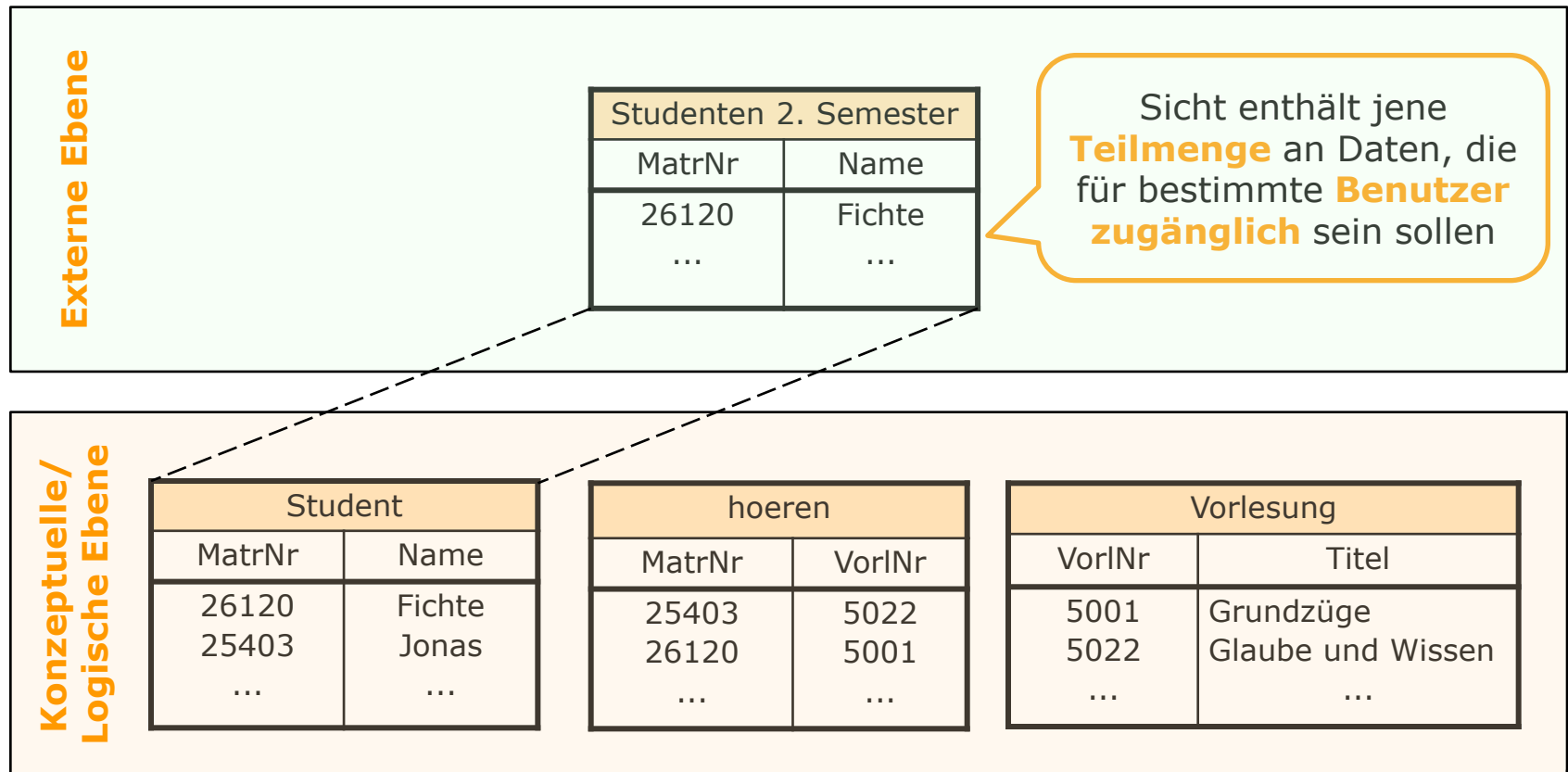


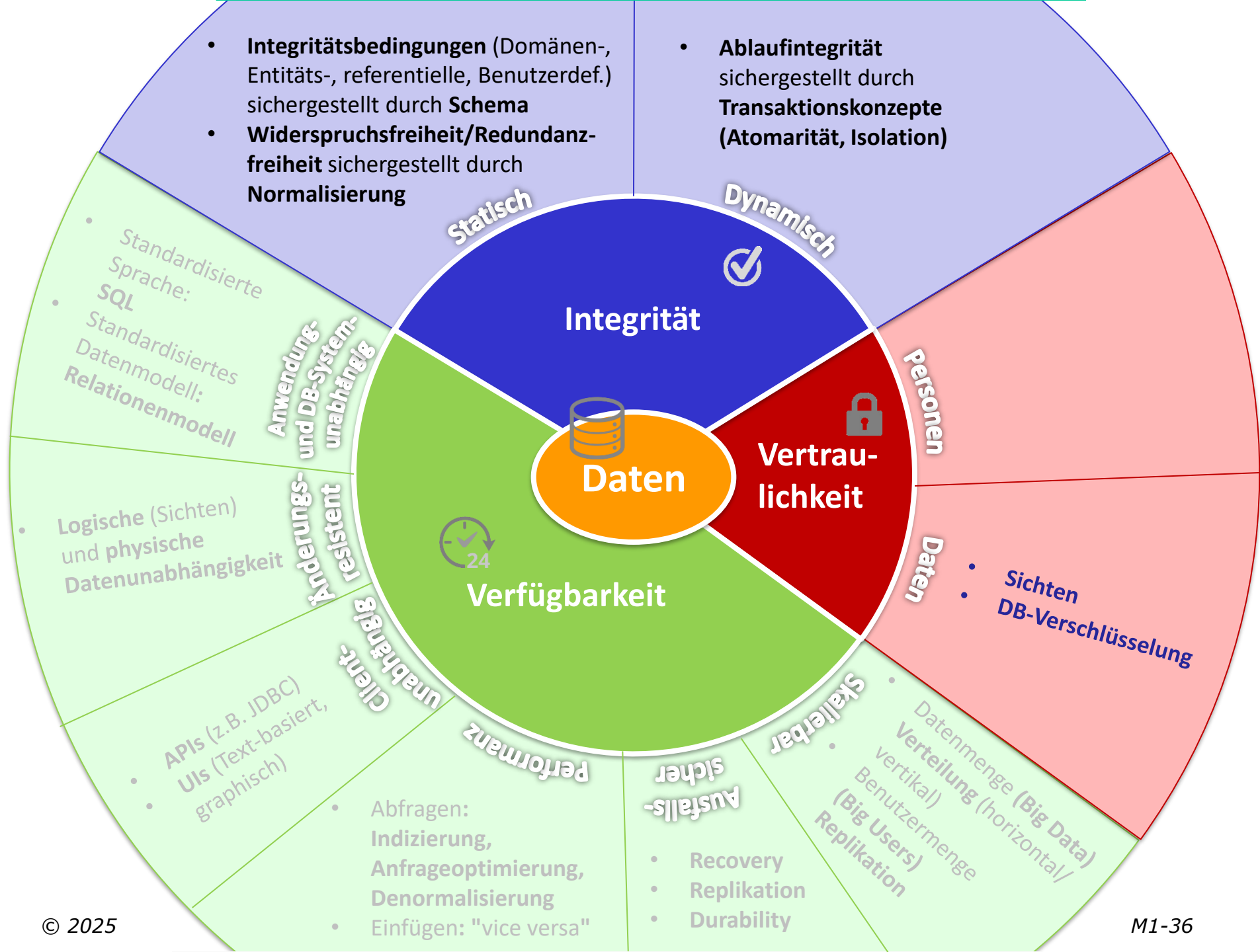
Aufgaben von Datenbanksystemen im Detail

Vertraulichkeit

■ Aus **Datensicht**

- Mit Hilfe von **Sichten** können Teile von Daten zugänglich gemacht werden und andere verborgen werden





Aufgaben von Datenbanksystemen im Detail

Vertraulichkeit

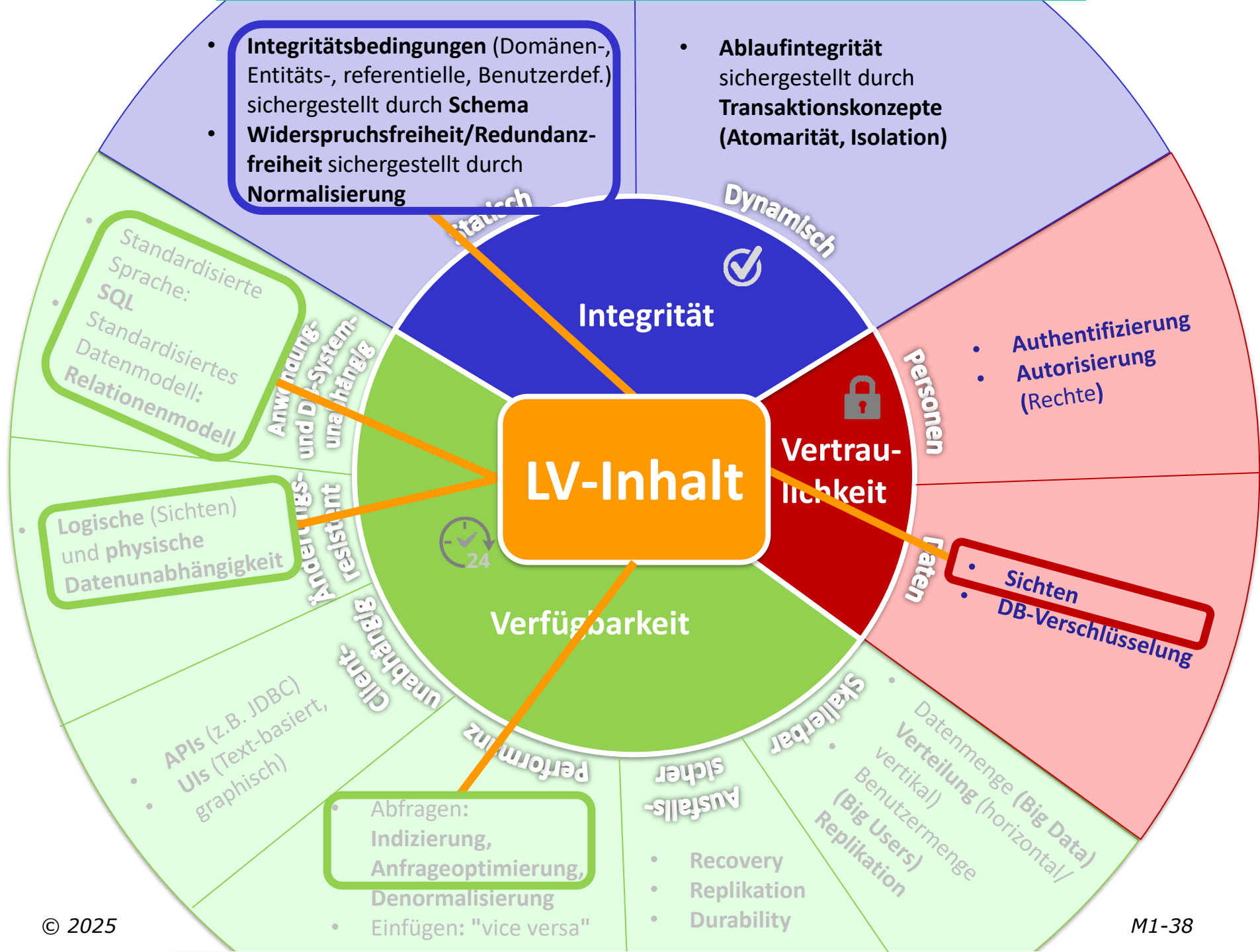
■ Aus Benutzersicht

- Mit Hilfe von **Rechten** kann z.B. pro Benutzer gesteuert werden, **welche Operationen** dieser Benutzer auf den **Daten** bzw. auf dem **DB-Schema** ausführen darf

Operationen auf
den **Daten**

Operationen auf
dem **DB-Schema**

Daten	Struktur	Administration
☒ SELECT	☒ CREATE	☐ GRANT
☒ INSERT	☒ ALTER	☐ LOCK TABLES
☒ UPDATE	☒ INDEX	☐ REFERENCES
☒ DELETE	☒ DROP	
	☐ CREATE TEMPORARY TABLES	
	☐ SHOW VIEW	
	☐ CREATE ROUTINE	
	☐ ALTER ROUTINE	
	☐ EXECUTE	
	☐ CREATE VIEW	



Ausblick Semester

■ Statische Datenintegrität

- Schema-Entwurf



■ Anwendungs- und DB-System-unabhängige Verfügbarkeit

- Relationenmodell
- SQL



■ Vertraulichkeit

- Sichten
- Rechte

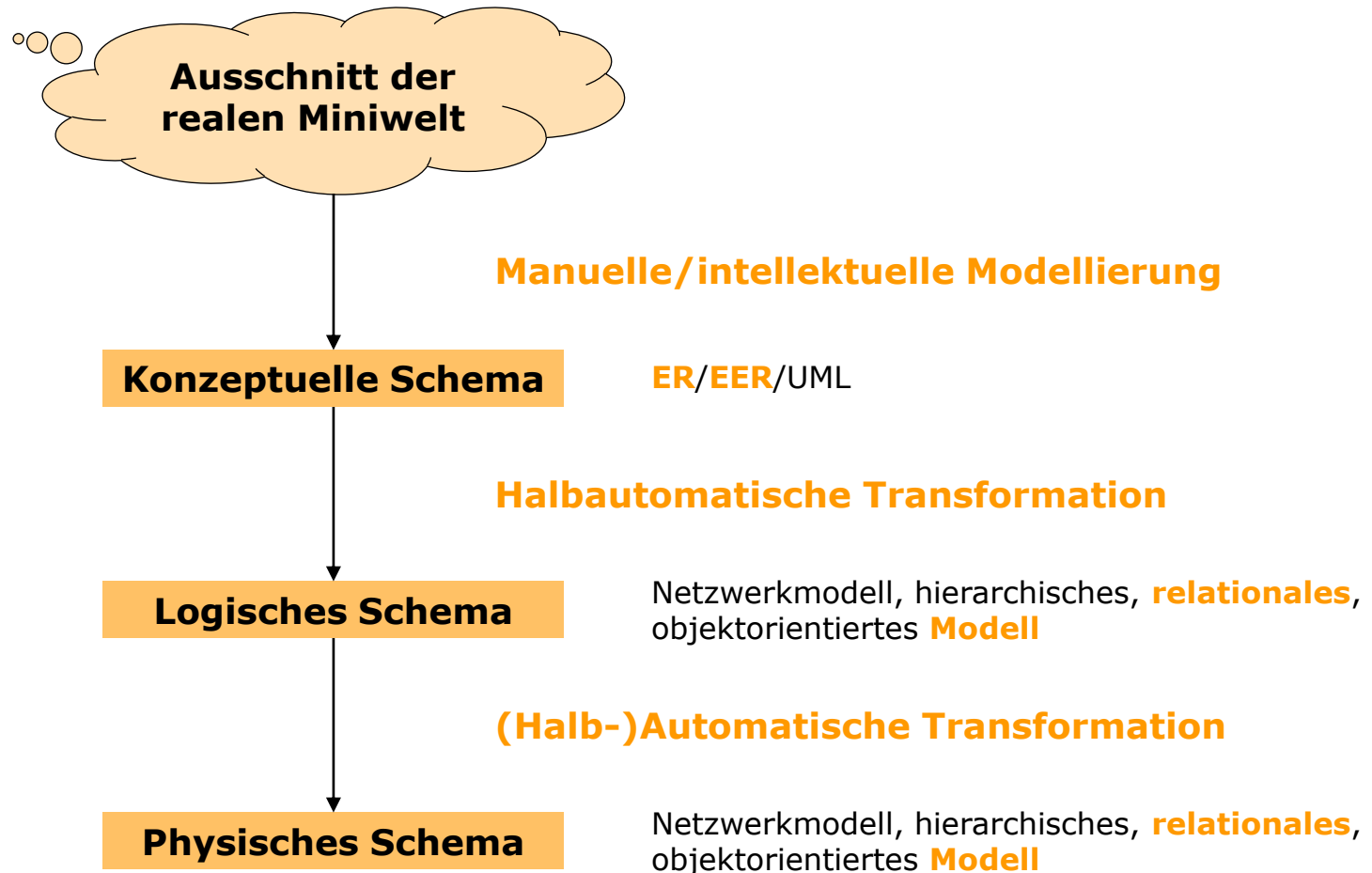


Sicherung
der Daten

Entwurf,
Aufbau und
Nutzung
von DB

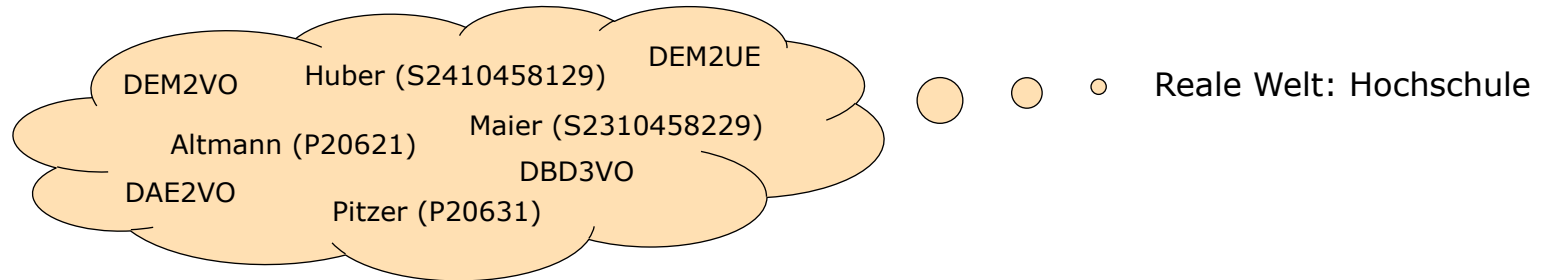
⇒ Unser **Fokus** liegt auf einer spezifischen Klasse von Datenbanksystemen: **Relationalen DB-Systemen**

Datenmodellierung 1/2

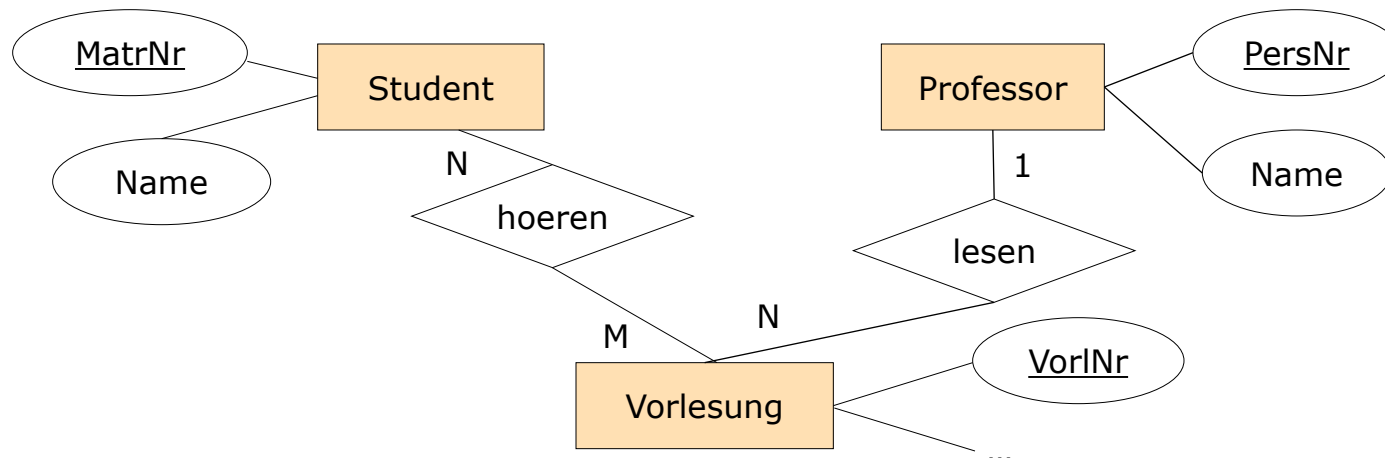


Datenmodellierung 2/2

① Abgrenzung der zu modellierenden Welt

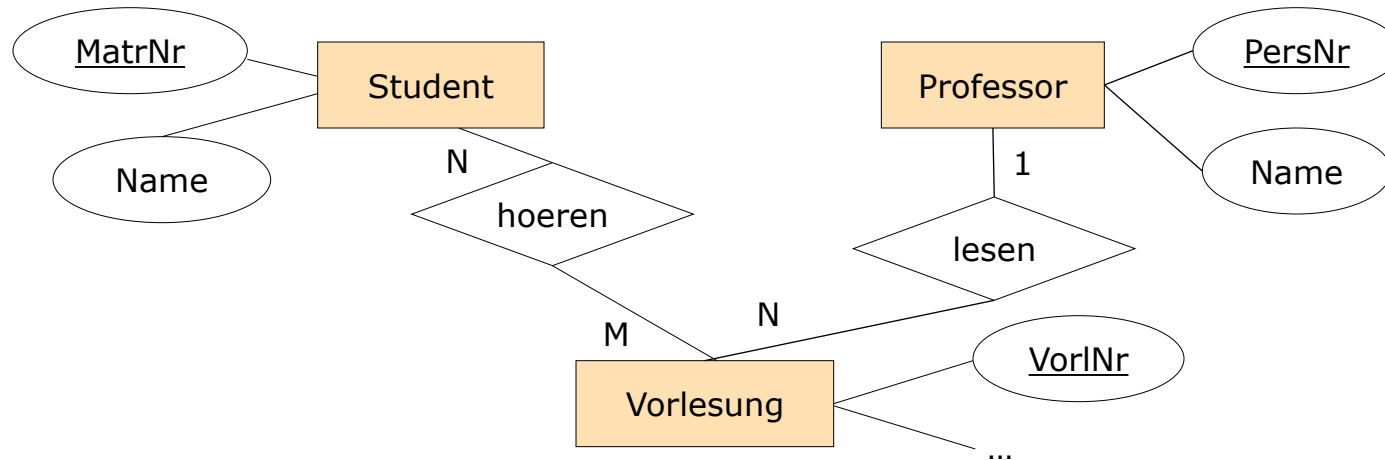


② Überführung der zu modellierenden Welt in ein konzeptuelles Schema (bspw. ER)



Datenmodellierung 2/2

② Konzeptuelles Schema



③ Überführung des konzeptuellen Schemas in ein logisches Schema (bspw. relationales Modell)

Student	
MatrNr	Name
...	...

hoeren	
MatrNr	VorlNr
...	...

Vorlesung	
VorlNr	Titel
...	...

Datenmodellierung 2/2

③ Logisches Schema (bspw. relationales Modell)

Student		hoeren		Vorlesung	
MatrNr	Name	MatrNr	VorlNr	VorlNr	Titel
...	...	...	...	...	...

④ Überführung des logischen Schemas in ein physisches Schema (bspw. relationales Modell)

```
CREATE TABLE Vorlesung (
  VorlNr NUMBER(10) PRIMARY KEY,
  Titel VARCHAR(20) NOT NULL,
  ...
);
```

Datenabfrage und -manipulation

Student	
MatrNr	Name
26120	Fichte
25403	Jonas
...	...

hoeren	
MatrNr	VorlNr
25403	5022
26120	5001
...	...

Vorlesung	
VorlNr	Titel
5001	Grundzüge
5022	Glaube und Wissen
...	...

```

SELECT Name
FROM Student, hoeren, Vorlesung
WHERE Student.MatrNr = hoeren.MatrNr AND
        hoeren.VorlNr = Vorlesung.VorlNr AND
        Vorlesung.Titel = 'Grundzüge';

```

```

UPDATE Vorlesung
SET Titel = 'Grundzüge der Logik'
WHERE VorlNr = 5001;

```

Überblick

- Einführung und Motivation
- Aufgaben von Datenbanksysteme
- Historische Entwicklung
- Entwicklungstrends
- Anhang I - IV: Definitionen

Historische Entwicklung: 60er Jahre

■ Anfang 60er Jahre

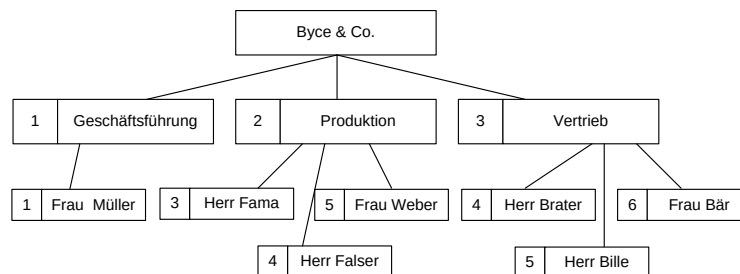
- elementare Dateien, anwendungsspezifische Datenorganisation
- geräteabhängig, redundant, inkonsistent

■ Ende 60er Jahre

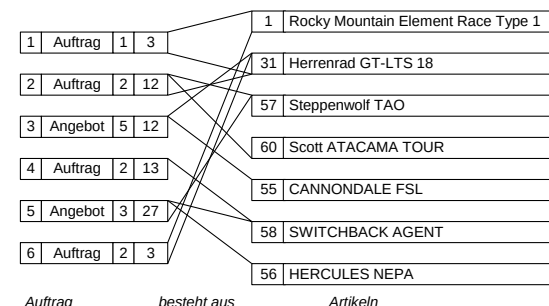
- Dateiverwaltungssysteme (SAM, ISAM) mit Dienstprogrammen
- geräteunabhängig, aber redundant und inkonsistent

■ DBS basierend auf hierarchischem Modell, Netzwerkmodell

- Zeigerstrukturen zwischen Daten
- Schwache Trennung interne / konzeptuell Ebene
- Navigierende DML / Datenzugriffspfade



Hierarchisches Modell (Bäume)



Auftrag besteht aus Artikeln

Netzwerkmodell (Graphen)

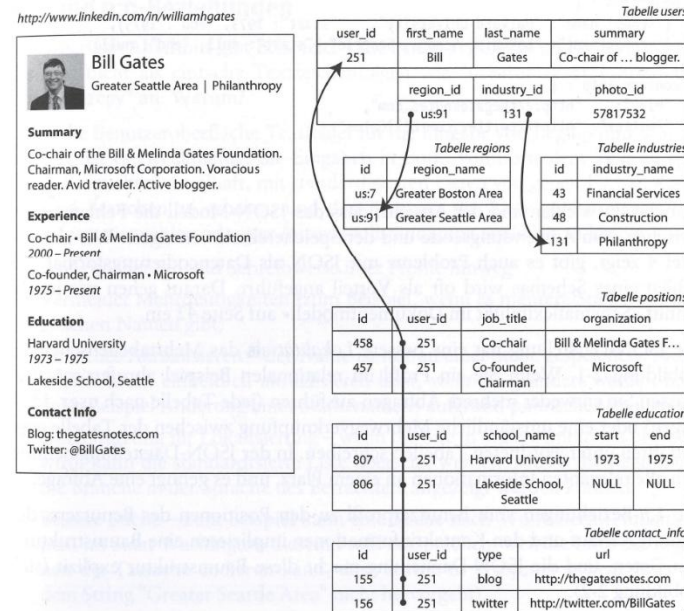
Historische Entwicklung: 70er u. 80er Jahre

■ 70er Jahre

- Datenbanksysteme
- Geräte- und Datenunabhängigkeit, **redundanzfrei**, konsistent

■ Relationale Datenbanksysteme

- Daten in Tabellenstrukturen
- Drei-Ebenen-Konzept
- Deklarative DML



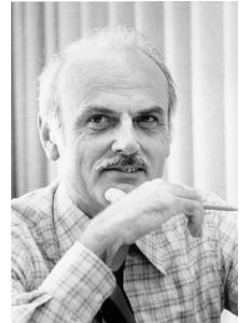
Relationale Datenbanksysteme

(vgl. Kleppmann M.: Datenintensive Anwendungen designen; O'Reilly, 2019)

Historische Entwicklung: Relationale DBS

ausgewählte Entwicklungen

- **1970**: E. F. Codd (IBM) $\Rightarrow$ Relationenmodell als konzeptionelle Grundlage relationaler DBS
- **1974**: System R (IBM) $\Rightarrow$ erster RDBMS-Prototyp
 - Anfragesprache SEQUEL ($\Rightarrow$ SQL)
 - erste Installation 1977
 - Vorgänger von DB2 und Oracle
- **1975**: University of California at Berkeley $\Rightarrow$ Ingres
 - Anfragesprache QUEL
 - Vorgänger von Postgres ($\Rightarrow$ PostgreSQL), Sybase, . . .
- **1979**: Oracle Version 2 (erstes kommerzielle DBS)
- **1989**: MS-SQL-Server (Quellcode von Sybase)
- **2009**: Oracle kauft Sun (Java, MySQL, ...)
- **2010**: SAP HANA (In-Memory-Datenbank)
- **2012**: Google Spanner (Cloud-basierte Datenbank)
- ...



Edgar Frank Codd
(Turing Award 1981)



Michael Stonebraker
(Neumann-Medaille 2005)

Historische Entwicklung: 90er Jahre

■ Objektorientierte Datenbanksysteme

- Daten in komplexeren Objektstrukturen (Trennung Objekt und seine Daten)
- Deklarative oder navigierende DML
- Oft integrierte Datenbankprogrammiersprache
- z.B. Poet, Versant, ObjectStore

■ Objektrelationale Datenbanksysteme

- Integration von objektorientierten Konzepten in bestehende relationale Datenbanksysteme
- z.B. Oracle, PostgreSQL

Historische Entwicklung: 2000er Jahre

- **Multimediadatenbanken**
 - Multimediale Objekte (Bilder, Audio, Video)
 - z.B. Oracle, PostgreSQL
- **XML-Datenbanken**
 - Verwaltung semistrukturierter Daten (XML-Dokumente)
 - z.B. TAMINO/Software AG
- **Verteilte Datenbanken**
 - Daten auf vielen (verschiedenen) Rechnerknoten
- **Mobile Datenbanken**
- **Datenverwaltung auf Kleinstgeräten**
- **Suche/Search**

Historische Entwicklung: 2010er Jahre

- Web-scale Daten
 - Das WWW selbst (Big Data)
 - Facebook, Google, Twitter, ...
- NoSQL („Not only SQL“)
 - Neue, einfachere DBMS
 - Hoch verteilt
- Spezialisierte DBMS
 - Keine eierlegende Wollmilchsau
 - Map/Reduce für Datenanalysen
 - Key-value-Speicher für Transaktionen
- Neue Hardware
 - Field Programmable Gate Array (FPGA), General purpose GPU (GPGPU), Solid state disks (SSD), Hauptspeicher

Überblick

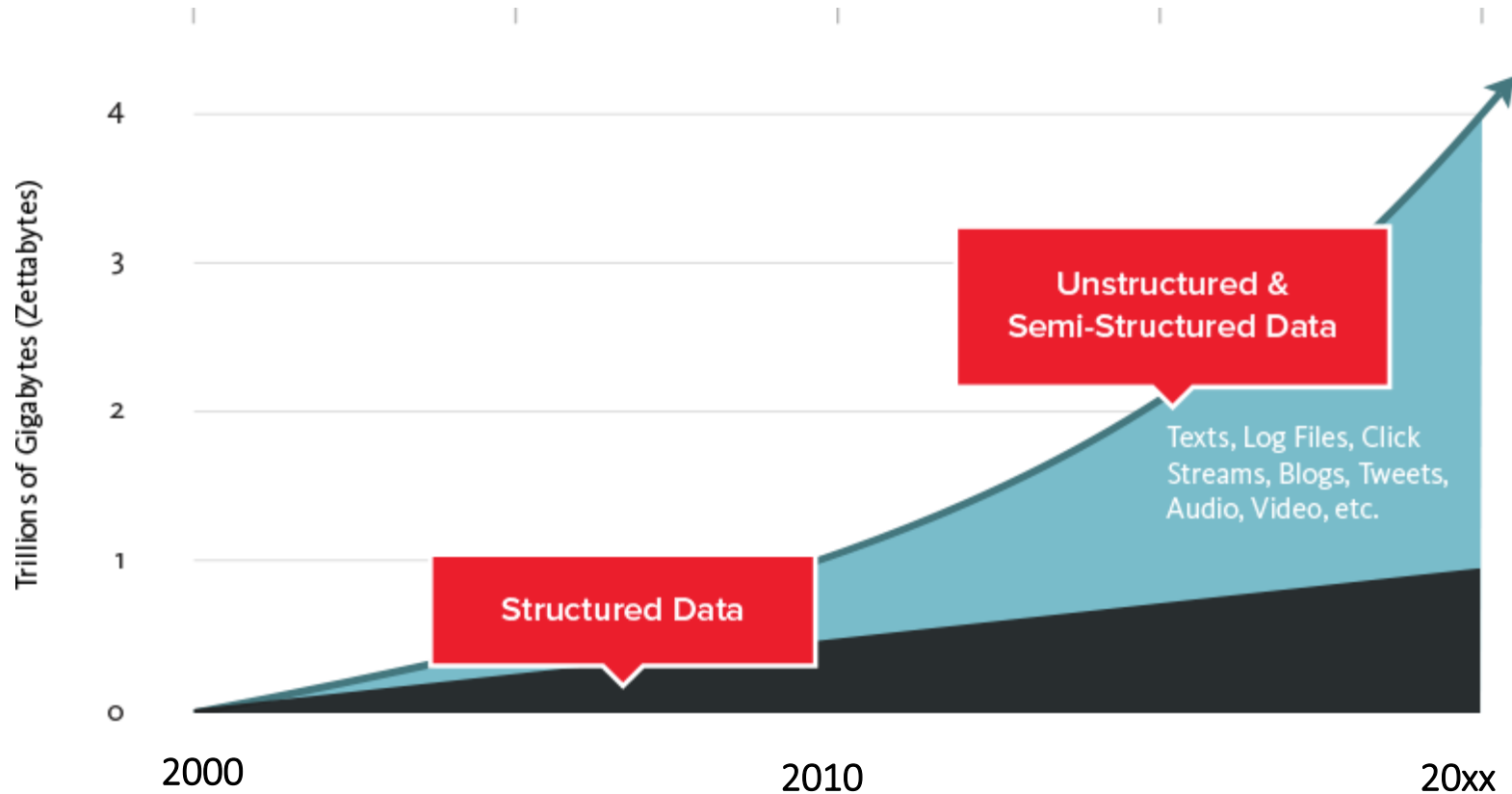
- Einführung und Motivation
- Aufgaben von Datenbanksystemen
- Historische Entwicklung
- Entwicklungstrends
- Anhang I - IV: Definitionen

Entwicklungslinien: heute

- **Multimediatatenbanken:**
 - Verwaltung multimedialer Objekte (Bilder, Audio, Video)
- **Verteilte Datenbanken:**
 - Verteilung von Daten auf verschiedene Rechnerknoten
- **Hochskalierbare, parallele Datenbanksysteme**
 - Umgang mit Datenmengen im Peta-Bereich
- **Mobile Datenbanken:**
 - Datenverwaltung auf Kleinstgeräten (PDA, Mobiltelefon, . . .)
- **NoSQL** Datenbanken
 - Google (Big Table), Oracle NoSQL, CouchDB, MongoDB
- **In-Memory** Datenbanken
 - SAP HANA, HSQLDB, VoltDB
- **NewSQL** Datenbanken
 - Google Spanner



Entwicklungstrend: NoSQL

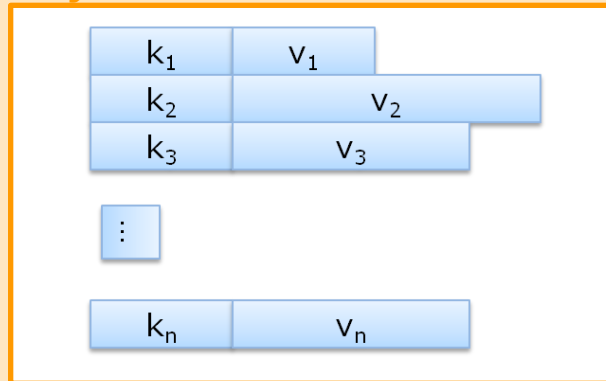


<http://www.couchbase.com/why-nosql/nosql-database>

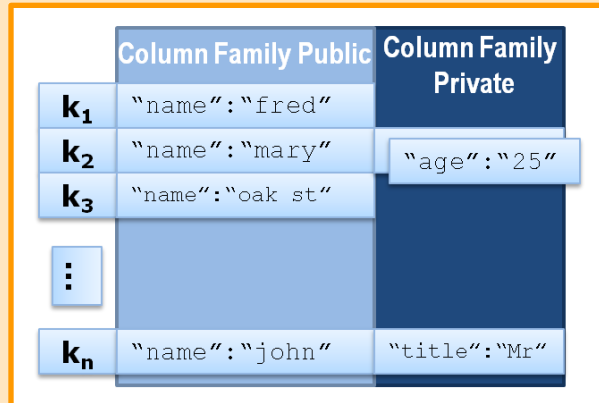
Entwicklungslinien: NoSQL

All Key-based!

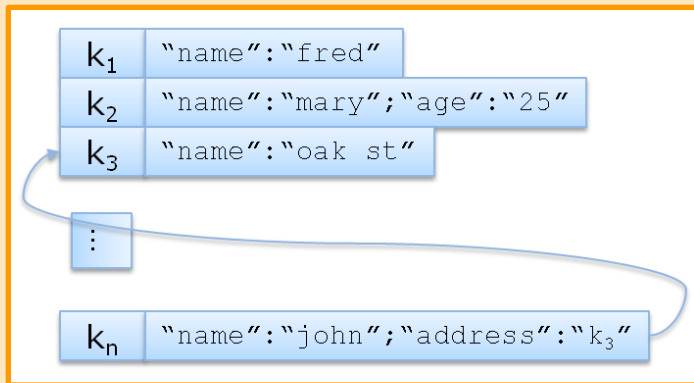
Key-Value



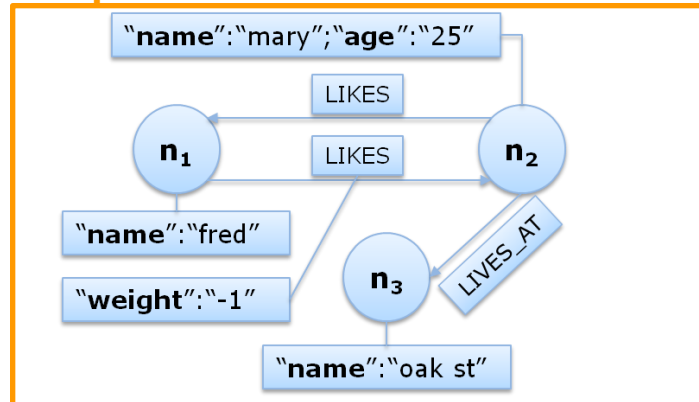
Wide-Column



Document



Graph-based



T. Haughey, The Role of Data Architecture in NOSQL, DAMA Day NY, Sept. 2011.
R. Cattell, "Scalable SQL and NoSQL Data Stores", 2010.

Entwicklungslinien: NoSQL

Key-Value

Key	Value
1	Bob
2	Sue
3	Joe
4	Jo



LinkedIn

Project Voldemort
A distributed database.



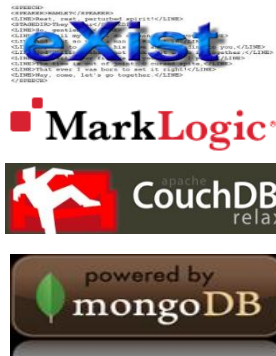
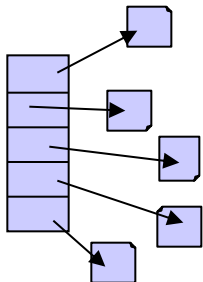
Wide-Column

	Col 1	Col2	Col3	Col4	Col5	Col6	Col7	Col8
Row 1								
Row 2								
Row 3								
Row 4								
Row 5								
Row 6								
Row 7								
Row 8								

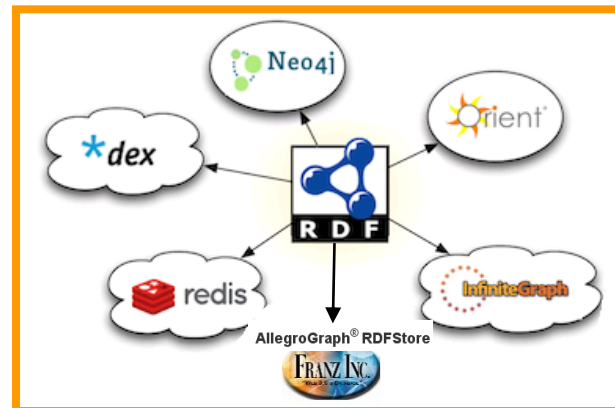
Google Bigtable



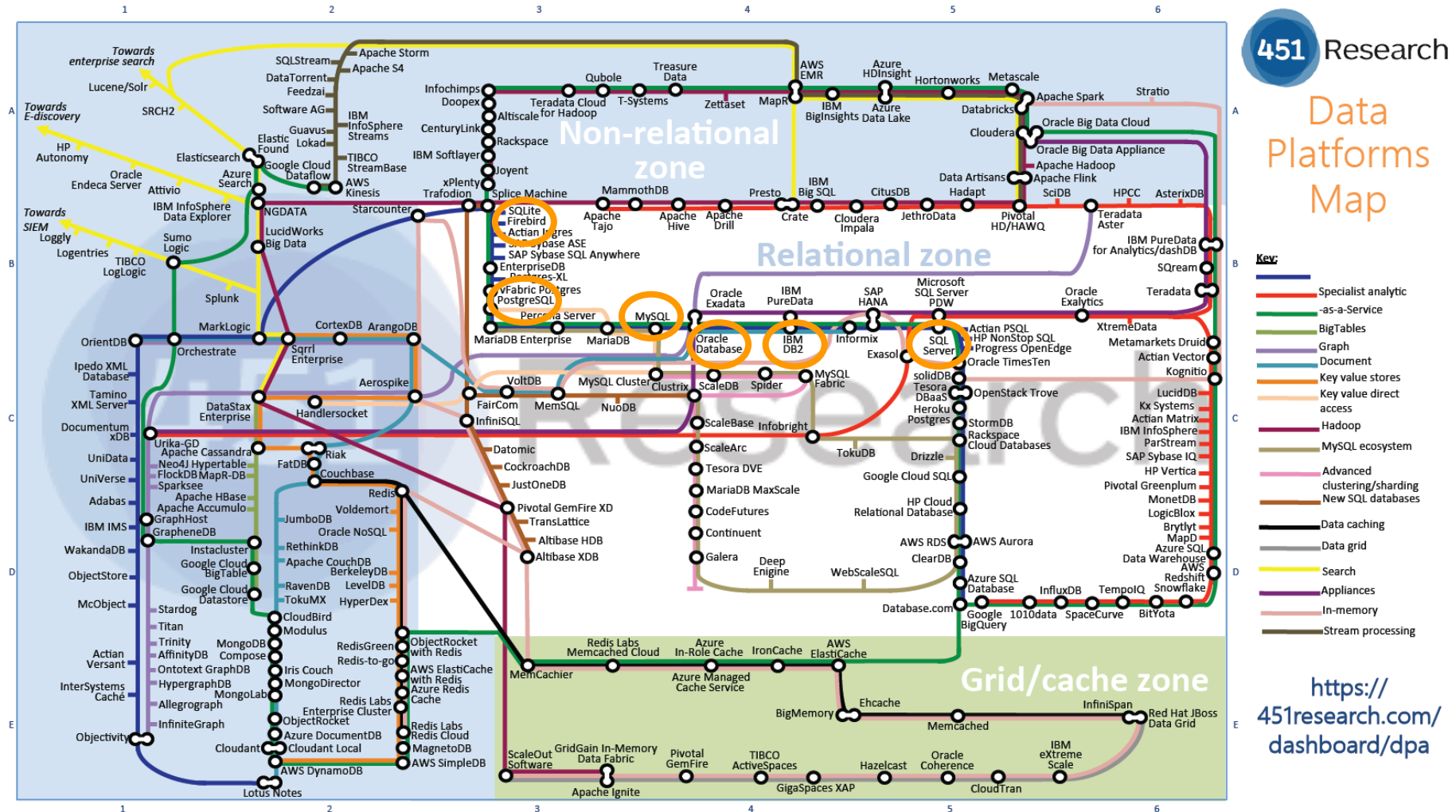
Document



Graph-based



Datenbank-Landschaft: heute



Anhang I: Definitionen

Datenbanksystem (DBS)

Ein Datenbanksystem (DBS) besteht aus einer oder mehreren Datenbanken (DB), einem Data Dictionary (DD) und einem Datenbankmanagementsystem (DBMS).

Datenbankverwaltungssystem (DBMS)

Ein Datenbankverwaltungssystem (Database Management System) bezeichnet die Gesamtheit der Programme zum Zugriff auf die im Datenbanksystem (DBS) gespeicherten Daten, zur Konsistenzkontrolle und zur Datendefinition.

Datenbank (DB)

Die in einem Datenbanksystem gespeicherten Daten werden als Datenbank bezeichnet.

Data Dictionary (DD)

Ein Data Dictionary (DD) enthält die Daten, die den Datenbestand beschreiben.

Anhang II: Definitionen

Datenmodell

Ein Datenbanksystem (DBS) basiert auf einem Datenmodell. Das Datenmodell stellt die Konzepte zur Modellierung der realen Welt zur Verfügung und besteht aus:

- Datendefinitionssprache (DDL)
- Datenmanipulationssprache (DML)
 - Abfragesprache (Query Language)
 - Befehle zum Einfügen, Löschen und Verändern der Daten

Anhang III: Definitionen

Datenbankschema (Intension)

Ein Datenbankschema legt die Struktur der Daten fest. (Metadaten: Daten über Datenbank)

Beispiel

Es gibt Studierende, die haben eine Matrikelnummer, einen Namen, ein Geburtsdatum. Zusätzlich gibt es Lehrende mit Sozialversicherungsnummer, Namen und Geburtsdatum.

Datenbankausprägung (Extension)

Die Datenbankausprägung stellt den momentan gültigen Zustand der Datenbank dar.

Beispiel

Es existieren 456 Lehrende, einer davon heißt Josef Altmann ...

Anhang IV: Definitionen

Redundanz

Redundanz ist das mehrfache Vorhandensein ein und derselben Informationen.

Konsistenz

Konsistenz bezeichnet in der Informatik allgemein die Widerspruchsfreiheit von Daten.

Datenintegrität

Unter Datenintegrität versteht man die logische Korrektheit, Gültigkeit und Genauigkeit von Daten in einer Datenbank.

Anhang V: Definitionen

Anomalie

Anomalien bezeichnen in relationalen Datenbanken Fehlverhalten der Datenbank. Anomalien entstehen durch einen fehlerhaften oder inkorrekten Datenbankentwurf.

Löschanomalie

Bei der Löschanomalie kann es passieren, dass man durch das bewusste Löschen eines Datensatzes, unbewusst Informationen verliert, die man später wieder gebraucht hätte.

Einfügeanomalie

Liegt ein fehlerhaftes Datenbankdesign vor, kann es bei der Einfügeanomalie passieren das Daten gar nicht gespeichert/angenommen werden, wenn beispielsweise für den Primärschlüssel kein Wert eingegeben wird, oder es führt bei einer nicht vollständigen Eingabe von Daten zu Inkonsistenz.

Änderungsanomalie

Bei der Änderungsanomalie werden gleiche Attribute eines Datensatzes nicht automatisch geändert. So entsteht eine Inkonsistenz der Daten. Man muss per Hand alle Einträge aktualisieren, es darf kein Fehler unterlaufen ansonsten führt es zur Inkonsistenz.